Introduction to Compilers



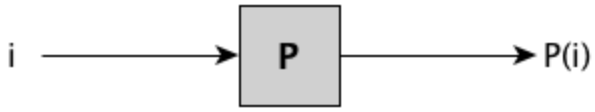
You know you want to write a compiler. Let's get oriented first. A short introduction will make your upcoming tasks easier and more enjoyable.

CONTENTS

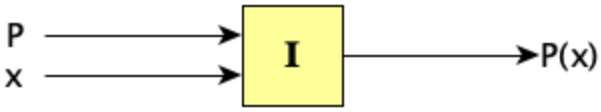
Programs, Interpreters and Translators • Writing Interpreters • Writing Translators • Compiler Writing in Practice • Why Should I Care? • Some Good Reads • Should I Take A Compiler Class? • Recall Practice • Summary

Programs, Interpreters and Translators

A **program** is a linguistic representation of a computation:



An **interpreter** is a program whose input is a program P and some input data x; it **executes** P on x:

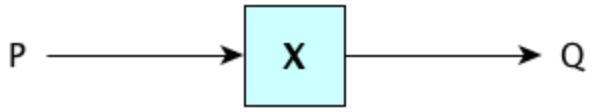


That is, $I(P, x) = P(x)$.

Interpreters can be implemented in hardware or software. For example, a CPU is a (hardware) interpreter for machine language programs.

> ***The Universal Interpreter*** 🤯🤯🤯
>
> *The fact that there exists a single interpreter capable of executing all possible programs (those representable in a formal system) is a very deep and significant discovery of Turing's. It was, and perhaps still, is, utterly mind blowing.*

It's too expensive (not to mention ridiculously difficult) to write interpreters for high-level languages, so we generally **translate** them into something more easily interpretable. A **translator** is a program which takes as input a program in some language S and outputs a program in a language T such that the two programs have the same **semantics**.



That is, $\forall x.\ P(x) = Q(x)$.

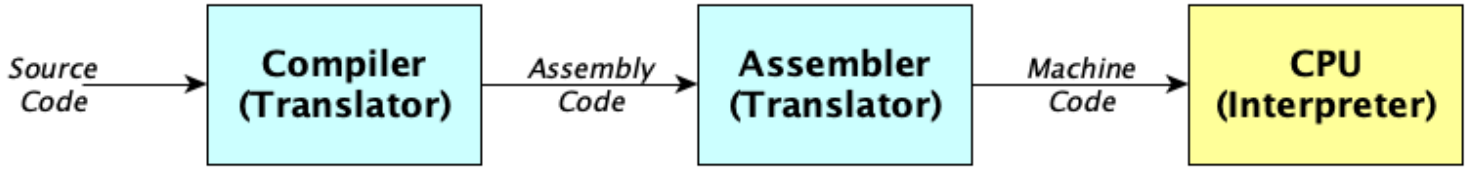
Some translators have really strange names:

Assembler
: A translator from assembly language programs to machine language programs.

Compiler
: A translator from "high-level" language programs into programs in a lower-level language.

Transpiler
: A translator from high-level language programs into programs in a different high-level language.

If we translate the entirety of a program into something interpretable, we call this ahead-of-time (AOT) compilation. AOT compilers may be chained, the last of which is often an existing assembler, like this:



When we compile portions of a program *while other previously compiled portions are running*, we have just-in-time (JIT) compilation. JIT compilers will often remember what they already compiled instead of compiling the same source over and over again. They can even do adaptive compilation and recompilation based on runtime behavior.

> ***It gets even more interesting***
>
> *It actually isn't true that compilation, whether AOT or JIT, is completely outside of your program. Many languages allow you to execute code during compilation time and compile new code at run time: think of macros in Julia or Clojure, Perl's BEGIN block, or any language with the dreaded* `eval` .

Writing Interpreters

Generally we write interpreters for simple, or low-level languages, whose programs tend to look like simple instruction sequences. As an example, let's imagine a simple programming language for evaluating and printing the results of arithmetic expressions. Here are three example programs:

- `8 P`
- `2 3 + N P`
- `3 1 + P 0 15 * 100 25 2 * / P P`

When you see a number, you push it on a stack. N negates the number on top. +, –, *, and / pop the two top elements, apply the operation, and push the result. P pops the value and prints it.

CLASSWORK

> Before we implement the interpreter, execute the three programs above by hand so you understand the operation.

You did the classwork, right? Great! Now let's write an interpreter:

stack-interpreter.js

```
function interpret(program) {
  const stack = []
  function push(x) {
    stack.push(Number(x))
  }
  function pop() {
    if (stack.length === 0) throw 'Not enough operands'
    return stack.pop()
  }
  for (const token of program.trim().split(/\s+/)) {
    let x, y
    if (/^\d+(\.\d+)?$/.test(token)) push(token)
    else if (token === 'N') (x = pop()), push(-x)
    else if (token === '+') (y = pop()), (x = pop()), push(x + y)
    else if (token === '-') (y = pop()), (x = pop()), push(x - y)
    else if (token === '*') (y = pop()), (x = pop()), push(x * y)
    else if (token === '/') (y = pop()), (x = pop()), push(x / y)
    else if (token === 'P') (x = pop()), console.log(x)
    else throw 'Illegal Instruction'
  }
}
```

Since the interpreter is a JavaScript program, you can just copy-paste it into your browser's console and invoke the function on any "program" of your choice, for example `interpret("8 P")`.

> ***Exercise**: Run this interpreter on several programs of your choosing.*

Writing Translators

When thinking about more sophisticated languages, we generally go through layers of translation before getting things down to something simple enough to interpret. That's fine, but we need to think deeply about what translation even is before attempting to implement a translator.

Translation Phases

First something inescapably true: translation consists of **analysis** and **generation** phases.

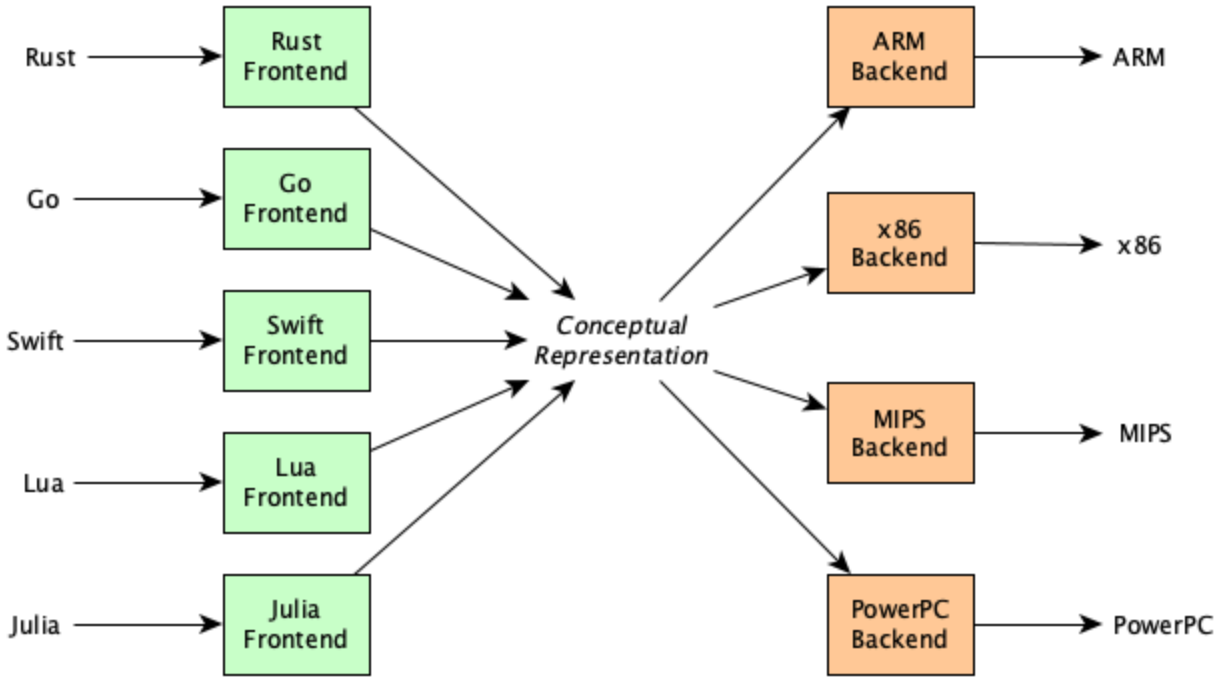


There are two reasons for this:

1. We can't really do it any other way. Word for word translation just doesn't work.

   > ***Exercise**: Give the most outrageous example you can think of where word-for-word translation between two natural languages comes out "so wrong."*

2. It makes good engineering sense: if we need to write translators for M source languages and N target languages, we need only write $M + N$ simple programs (half-compilers) rather than writing $M \times N$ complex programs (full-compilers).



In practice it is rare for a conceptual representation to exist that is expressive enough and powerful enough to capture every imaginable source language and every imaginable target. Some systems do

manage, at least, to get a lot of the common ones.

Natural Language Translation

How successful is machine translation for natural languages? Might it work best when the two languages are close cousins or siblings in an evolutionary language tree? Whether or not that's the case, it's often illuminating to play **translation telephone**. Here is one run I made using Google Translate:

1. (English) She's feeling under the weather so will work from home today, with a laptop and cup of tea
2. (Afrikaans) Sy gevoel onder die weer so sal werk van die huis af vandag, met 'n skootrekenaar en koppie tee
3. (Hindi) वह मौसम के अंतर्गत महसूस किया तो एक लैपटॉप और एक कप चाय के साथ आज घर से काम करेंगे
4. (Bulgarian) Той не се чувства добре с лаптоп и чаша чай ще работят от дома си днес
5. (Filipino) Hindi niya pakiramdam komportable sa isang laptop at isang tasa ng tsaa ay trabaho mula sa bahay sa araw na ito
6. (Sudanese) Anjeunna teu ngarasa nyaman jeung laptop jeung cangkir tea gawe ti imah ayeuna
7. (Kazakh) Ол бар ноутбук жайлы сезінетін жоқ, бір кесе шай бүгін үйден
8. (Chinese) 他不舒服的感覺與筆記本電腦，從家裡杯茶今天的工作
9. (Telugu) ల్యాప్టాప్ తన అసౌకర్యం, నేడు ఇంటి నుండి పని, టీ ఒక కప్పు
10. (English) His discomfort with a laptop, working from home today, a cup of tea

With idioms, sayings, and slang, you only need two languages to get something funny (the following really happened on Google Translate, but has since been fixed):

1. (English) later alligator
2. (Arabic) تمساح بيلحقك اركضي ولي هبلة
3. (English) A crocodile hit you with my knees

"Screenshot or it didn't happen":





> ***Exercise**: Run some translation telephone iterations of your own. Use English language idioms. Share your own funny results online.*

Machine translation for natural languages is getting much better. They use a lot of statistical methods, machine learning, dictionaries and lots of interesting techniques. We're looking for great translations, because perfection is not always possible.

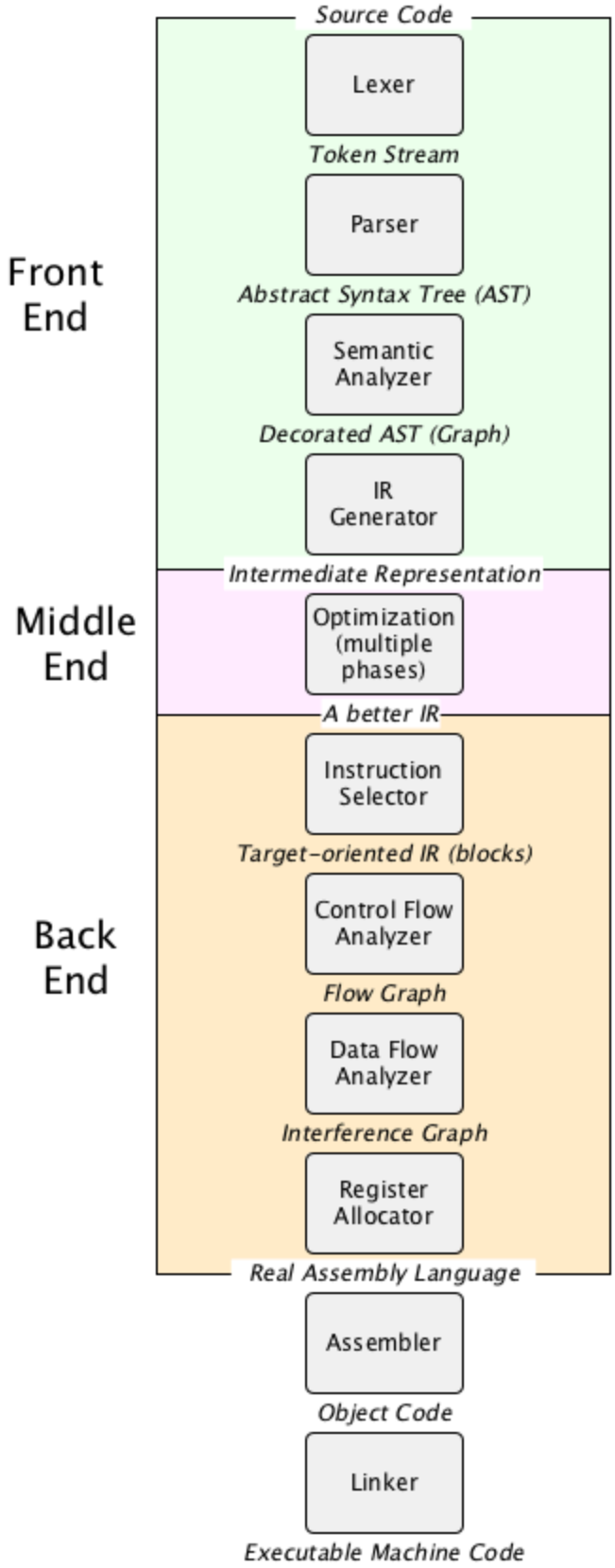
Fun read: Enjoy this article from December, 2016, about issues in natural language translation, and when computers started to get really good at it.

> ***Exercise**: Do some research into some major challenges still faced in natural language translation. One that comes to mind is gendered pronouns (as illustrated in the above telephone example). What kind of strategies can be employed in such cases?*

Programming Language Translation

Programming languages tend to be free of the ambiguities and nuances of natural languages, so translations can be in most cases 100% precise, and even formally verified. How do these systems look in practice? Decades of research into translation techniques have yielded some useful knowledge.

Compilers for programming languages go through a lot of phases, some have several dozen. In the so-called “classic compiler” (a standalone compiler that does everything from scratch), you tend to see the following major phases:



What do these phases look like in practice? Here’s a super-simplified example, just to get a taste for what is going on, using this C code:

```
#define ZERO 0
    unsigned  gcd(   unsigned   int  // Euclid's algorithm
      x,unsigned   y) {   while ( /* hello */  x>   ZERO
   ){unsigned oldx=x;x=y   %x;y  = oldx ;}return y ;}
```

Lexical Analysis

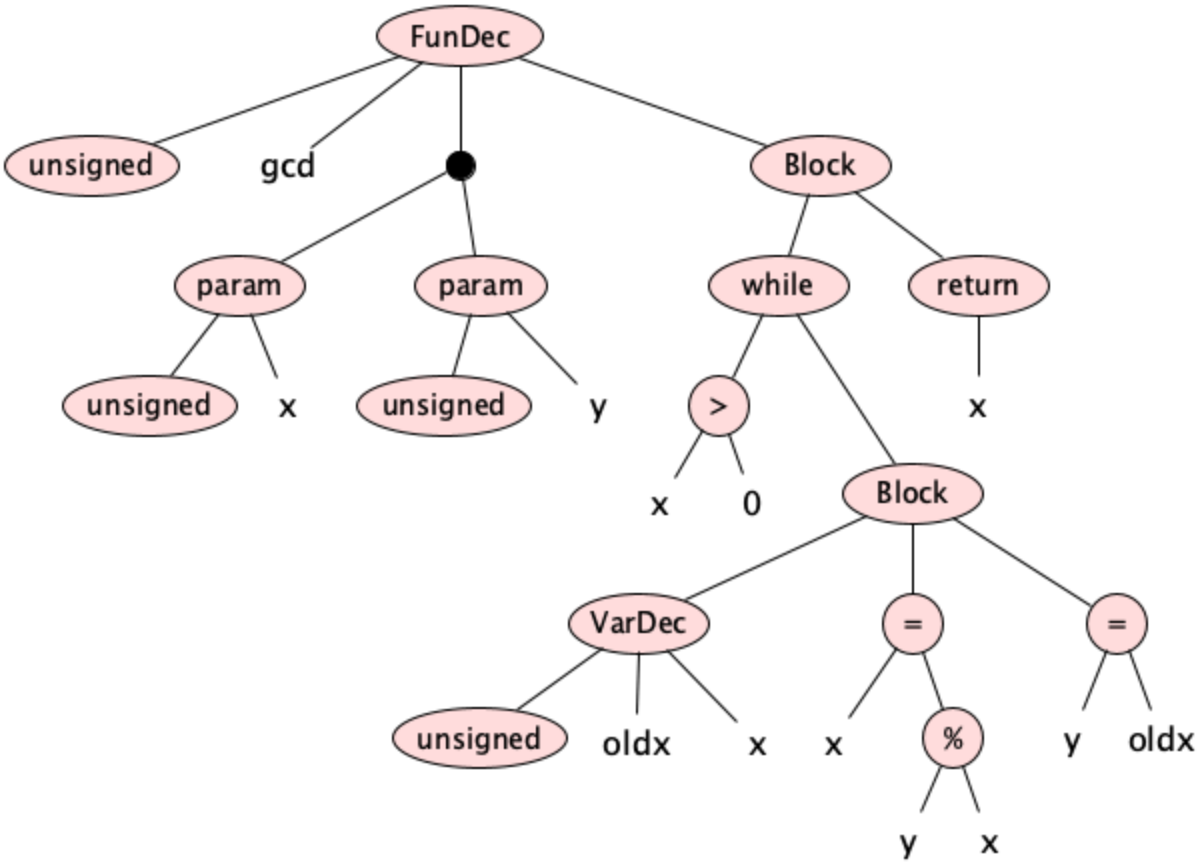
The **lexer** converts the source character stream into **tokens**. In doing this it needs to do things like remove comments, expand source-level macros, deal with indentation (for whitespace-significant languages like Python and Haskell), and remove whitespace. For the C code above, the lexical analyzer produces the token sequence:

`unsigned` `ID(gcd)` `(` `unsigned` `int` `ID(x)` `,` `unsigned` `ID(y)` `)` `{` `while` `(` `ID(x)` `>` `INTLIT(0)` `)` `{` `unsigned` `ID(oldx)` `=` `ID(x)` `;` `ID(x)` `=` `ID(y)` `%` `ID(x)` `;` `ID(y)` `=` `ID(oldx)` `;` `}` `return` `ID(y)` `;` `}`

To build a token stream, lexers have to take into account case sensitivity, whitespace and newline significance, comment nesting, the actual source language alphabet, any weird limits on identifier lengths. They need to be able to detect whether character or string literals are unclosed, or whether a file ends while inside a comment. If a token stream cannot be formed, the lexer emits a **lexical error**.

Parsing

Next, the **parser** applies the grammar rules to the token sequence to produce the **abstract syntax tree**. In our running example, the AST is:



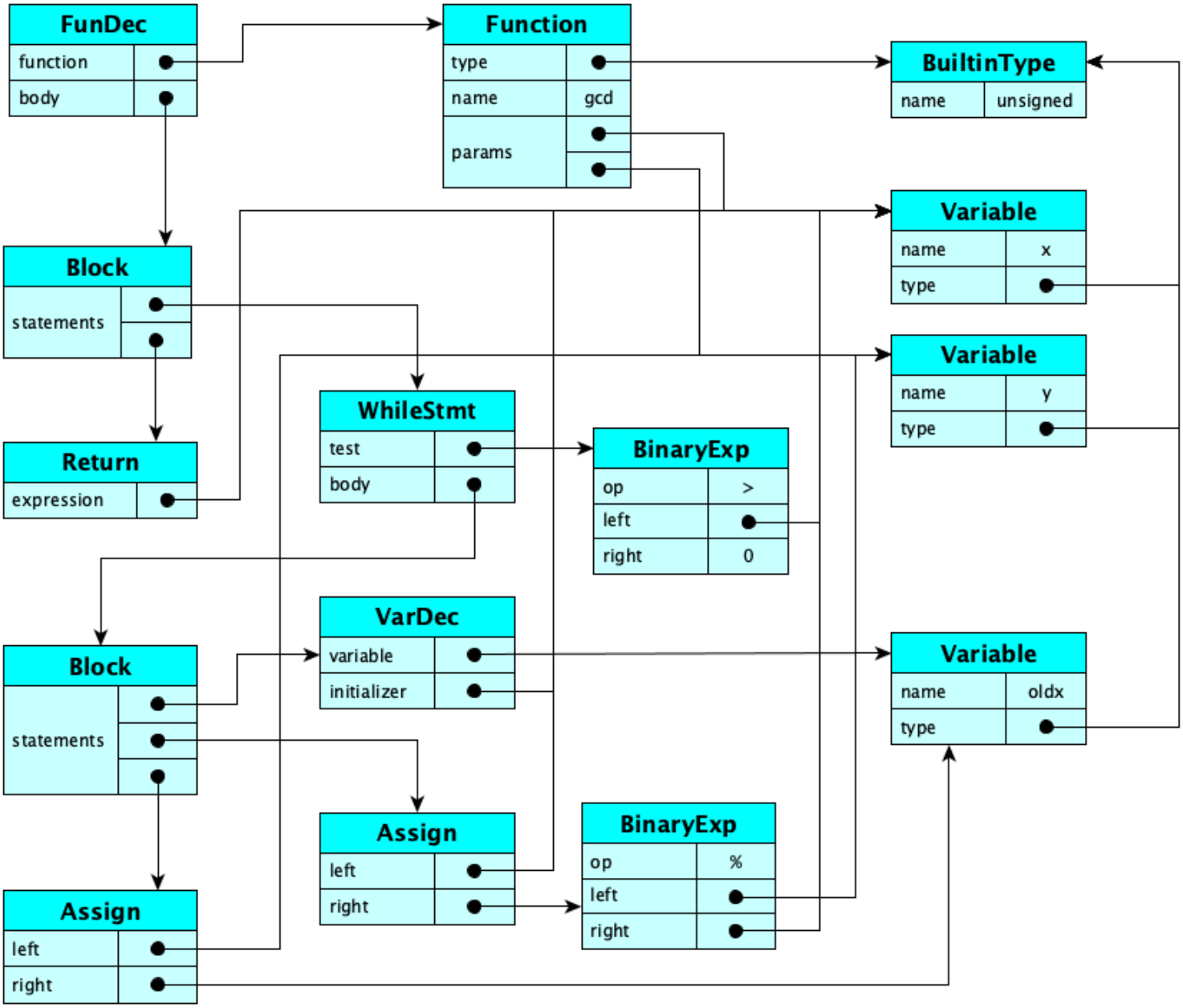
Parsers, like lexers, are often automatically generated from a grammar by special tools, called parser generators, but you can write them by hand if needed.

Errors that occur during parsing, called **syntax errors** arise from malformed token sequences such as:

- `j = 4 * (6 - x;`
- `i = /5`
- `42 = x * 3`

Static Analysis

Programming language grammars are almost always context free, so things like multiple declarations of a variable within a scope, referencing a variable before its (or without a) declaration, access violations, too many or too few arguments, or type checking failures, are checked after grammar matching, by the **static analyzer**. This produces the conceptual representation of the front end, which for our running example, is:



Intermediate Code Generation

Usually, an **intermediate code generator** transforms the graph representation of the program into something "flatter" like an abstract instruction list. For our running example, something like:

```
L1:
  if x > 0 goto L2
  oldx := x
  t1 := y % x
  x := t1
  y := oldx
  goto L1
```

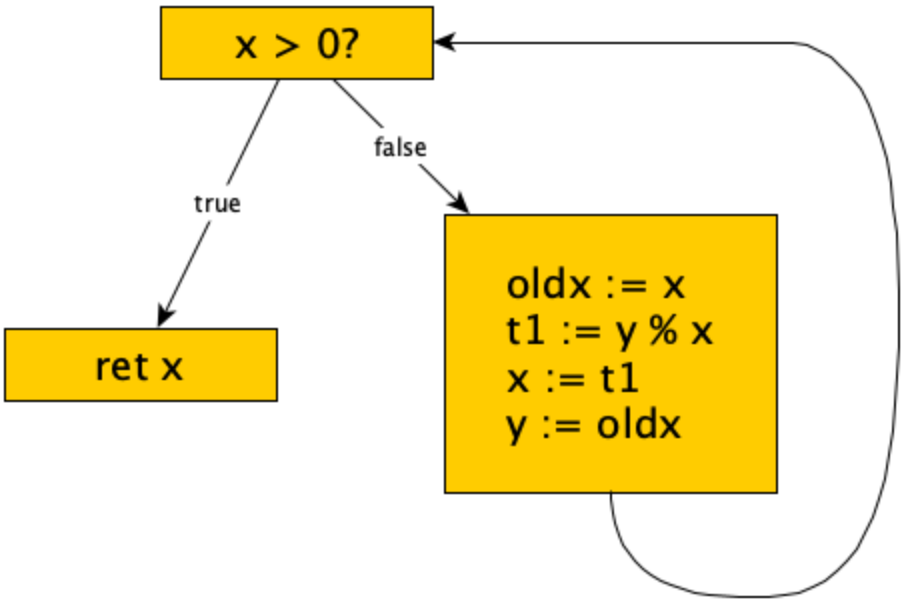
```
L2:
  return y
```

Here t1, t2, ... are **virtual registers**. At this stage of compilation, we assume an infinite number of these.

Also at this stage, we can look for an apply various **optimizations** such as constant folding and strength reductions.

Control Flow and Data Flow Analysis

The intermediate code is often broken up into chunks known as **basic blocks** and assembled into a **flow graph**. Here's our example:



We use the flow graph for control flow and data flow analysis, which helps with fancier optimizations like copy propagation, dead code elimination, induction variable simplification, loop fission and fusion, and more.

Code Generation

Classic compilers target machines with limited registers and fancy instruction sets, which have all sorts of arcane conventions involving register usage, memory allocation to manage function calls, and so much more. Code generators need to have intimate knowledge of processor architecture, so they can generate good code. In practice, code generators can be tuned to produce raw code (that is as close as possible to the source code) or "optimized" code. Here is unoptimized and optimized code for our running example on the ARM processor:

```
gcd(unsigned int, unsigned int):
        sub     sp, sp, #32
        str     w0, [sp, 12]
        str     w1, [sp, 8]
.L3:
        ldr     w0, [sp, 12]
        cmp     w0, 0
        beq     .L2
        ldr     w0, [sp, 12]
        str     w0, [sp, 28]
        ldr     w0, [sp, 8]
        ldr     w1, [sp, 12]
        udiv    w2, w0, w1
        ldr     w1, [sp, 12]
        mul     w1, w2, w1
        sub     w0, w0, w1
        str     w0, [sp, 12]
        ldr     w0, [sp, 28]
        str     w0, [sp, 8]
        b       .L3
.L2:
        ldr     w0, [sp, 8]
        add     sp, sp, 32
        ret
```

```
gcd(unsigned int, unsigned int):    ; x in w0, y in w1
        cbz     w0, .L2             ; if x == 0 goto L2
.L3:
        udiv    w2, w1, w0          ; w2 := y / x
        msub    w2, w2, w0, w1      ; w2 := y mod x
        mov     w1, w0              ; y := x
        mov     w0, w2              ; x := w2
        cbnz    w2, .L3             ; if x != 0 goto top
.L2:
        mov     w0, w1              ; return y
        ret
```

Compiler Writing in Practice

Now how many of these phases do people really write? Certainly not all of them, right? Aren't there tools that can help?

Unless you are a student or hobbyist looking for a deep understanding of compiler technology, you don't duplicate all the hard work that people have already put into middle and back ends. Most people these days make their front end target the LLVM system, which supplies middle and backend functionality for a large number of architectures. LLVM is so popular these days that Andi McClure created a helpful diagram for y'all:

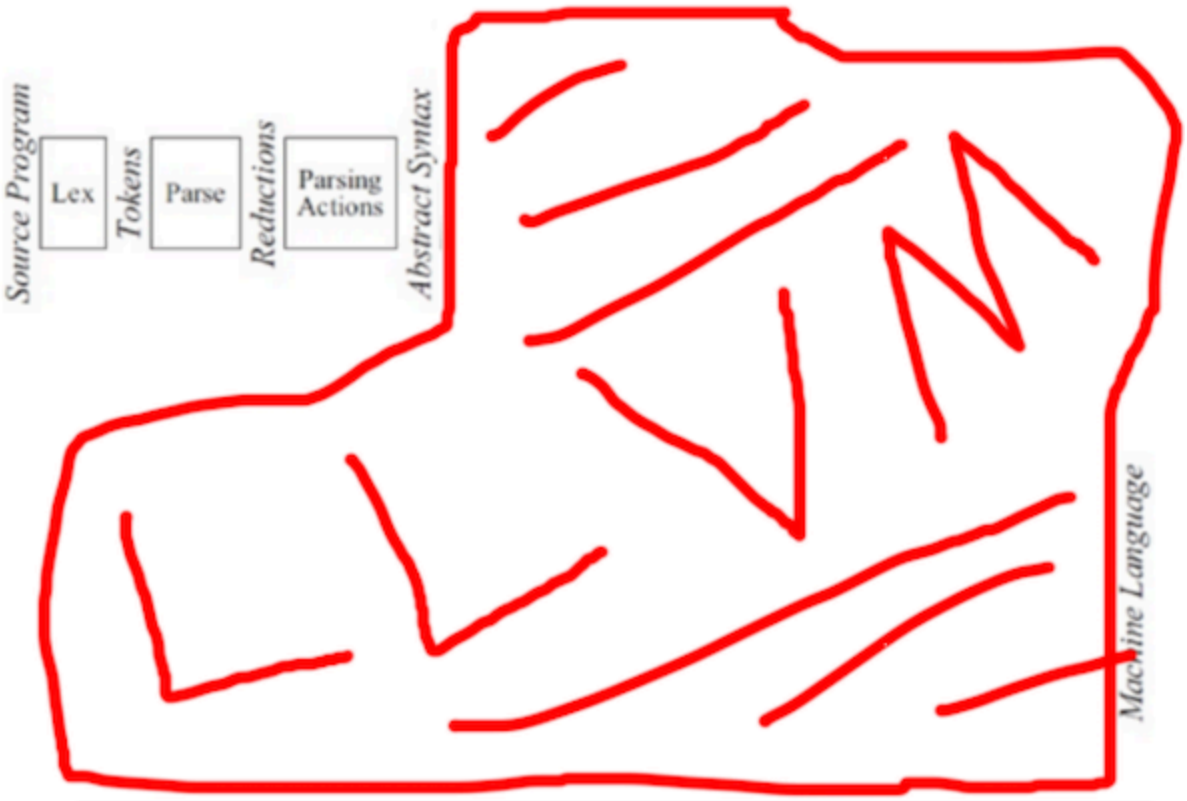
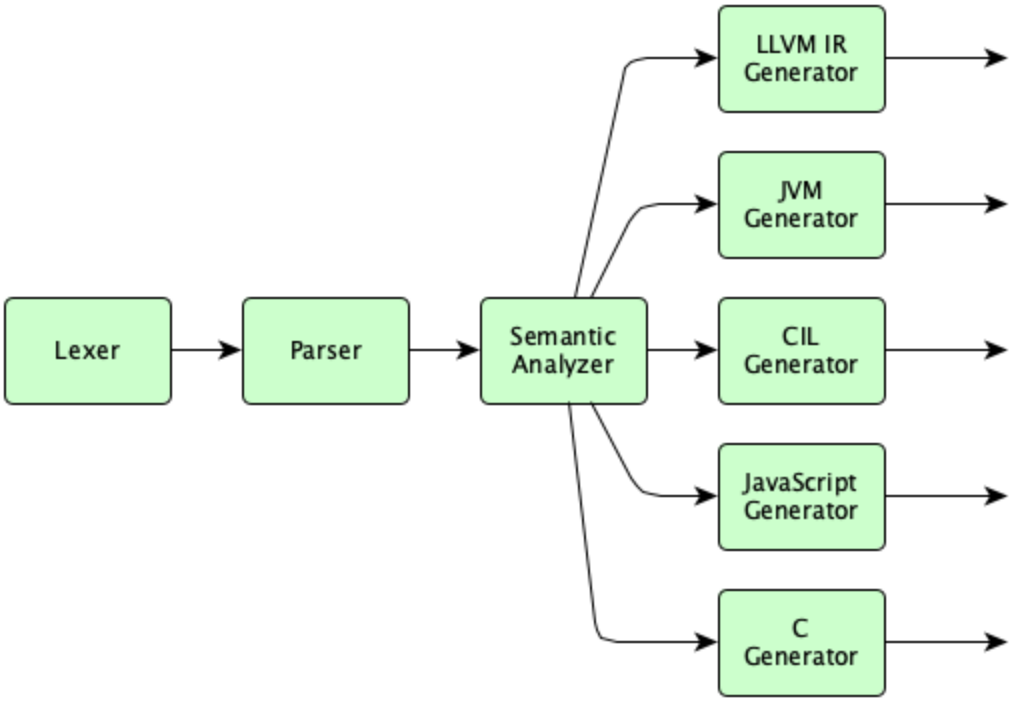


FIGURE 1.1. Phases of a compiler, and interfaces between them

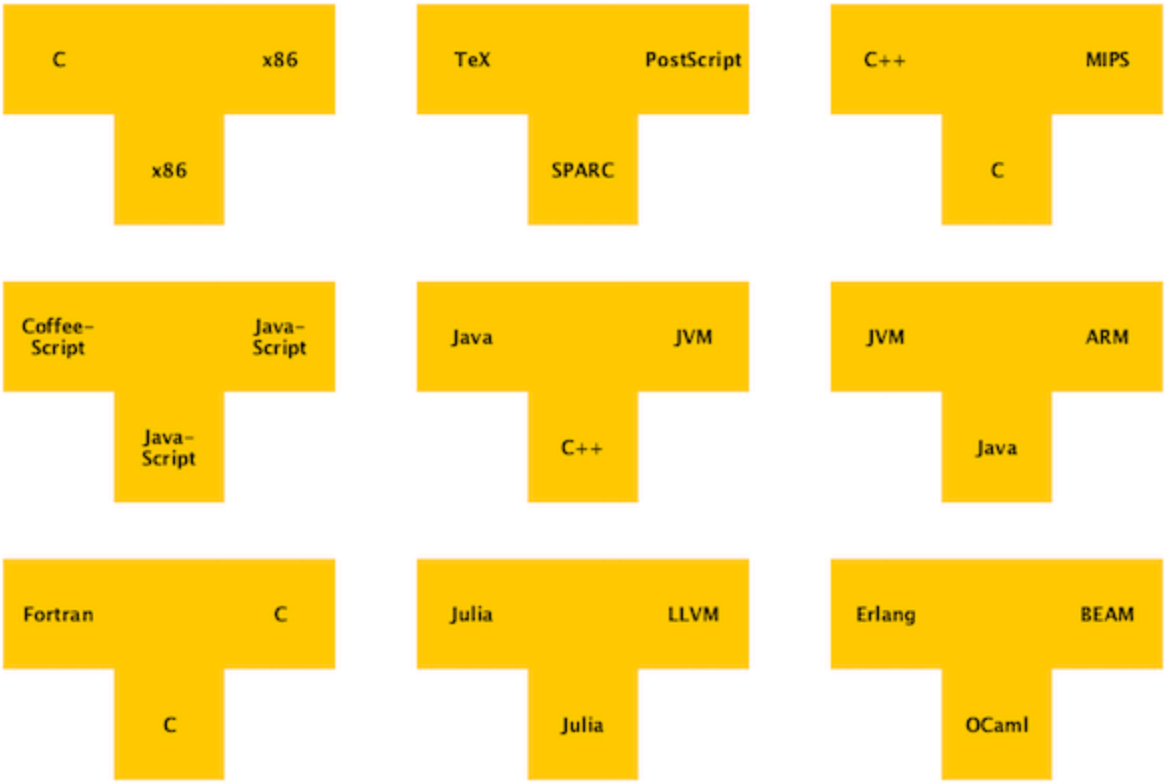
You don't have to target LLVM of course. You can target other systems like JVM (the Java Virtual Machine) if you want to interoperate with the massive Java ecosystem, or CIL, an intermediate language on the .NET platform. Heck, you can even simply target C or JavaScript and take advantage of the amazing existing JavaScript engines and existing C compilers others have created already.



You might think just hitting C is the best. But why is LLVM so popular? LLVM is more than just a set of compiler phases, it's a huge library of reusable components that allow all kinds of language processing. In fact, it's generally preferable to use LLVM when implementing a language rather than simply translating to C:

> *... translate the input source to C code (or some other language) and send it through existing C compilers. This allows reuse of the optimizer and code generator, gives good flexibility, control over the runtime, and is really easy for front-end implementers to understand, implement, and maintain. Unfortunately, doing this prevents efficient implementation of exception handling, provides a poor debugging experience, slows down compilation, and can be problematic for languages that require guaranteed tail calls (or other features not supported by C).* — Chris Lattner, from *the LLVM chapter in The Architecture of Open Source Applications*

There are *three* languages involved in translation: the source, the target, and the host. We show them in T-shapes, with source on the left, target on the right, and host below. Examples:



Here are three kinds of compilers you should learn to identify:

- Source language = Host language ⇒ **Self compiler**
- Target language = Host language ⇒ **Self-resident compiler**
- Target language ≠ Host language ⇒ **Cross compiler**

> ***Exercise**: Work out, or research, then explain how to use a self-compiler and a cross-compiler to port a compiler to an architecture that has no compiler.*

> ***Exercise**: A compiler is called an **optimizing compiler** if it works really hard to produce the most efficient target code that is feasibly possible. What can you use an optimizing self-compiler for?*

Let's See Some Examples

You'll want to study existing compilers to get a feel for what's involved. There are texts and tutorials about writing little languages, and there are real compilers for industrial-strength languages. Interested in the latter? Take a look at these:

- **Rust**. The Rust Playground (a.k.a. Rust Playpen). Type your code into a box and hit either the LLVM or ASM buttons.

- **TypeScript**. The TypeScript Playground. Your TypeScript code is compiled into JavaScript as you type!
- **C**, **C++**, **Fortran**, **Ada**, or any language compilable by GCC. Compile with `gcc -S` and view the generated assembly language in a `.s` file.
- **Julia**. Command line: `julia --output-bc hello.jl` to see the LLVM.
- **Swift**. Command line:
  - `swiftc -emit-sil hello.swift` for Swift Intermediate Language
  - `swiftc -emit-ir hello.swift` for LLVM
  - `swiftc -emit-assembly hello.swift` for the assembly language
- **Go**. Command line: `go tool compile -S hello.go` shows you the assembly language.
- **Java**. Command line: `javap -c Hello` shows you JVM code in the class file `Hello.class`.

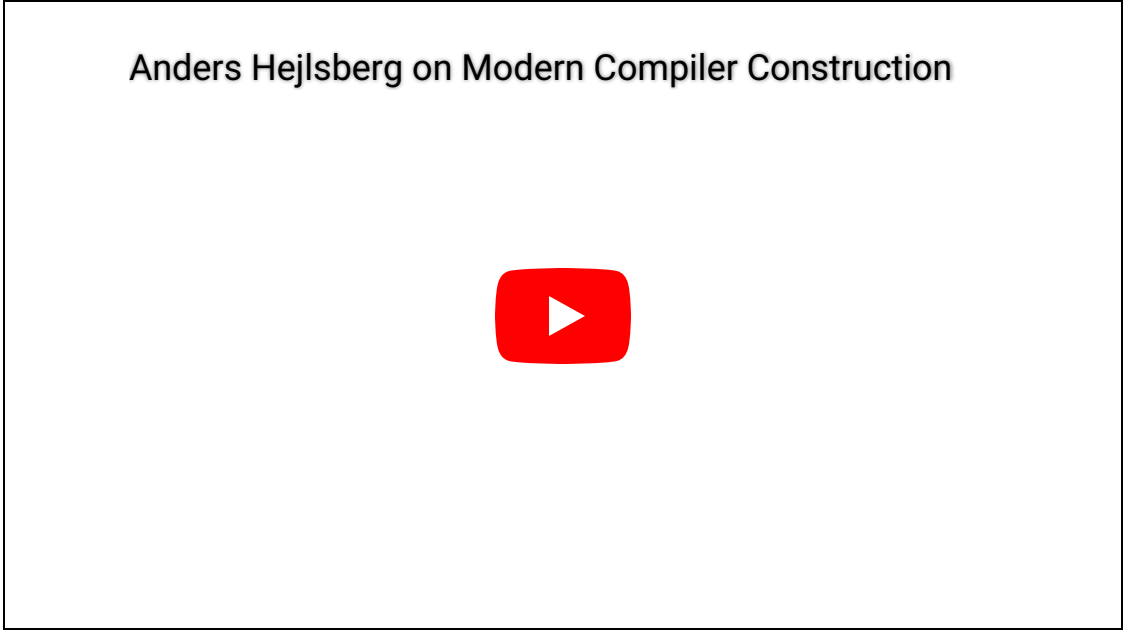
> ***Exercise**: Experiment with at least three of the above.*

And don't miss the AWESOME Compiler Explorer which does live translations from C++, Rust, Go, and D, to x86 assembly and machine code.

> ***Exercise**: Experiment with the Compiler Explorer.*

But wait, it's not just about the compiler!

Writing a traditional compiler will enter you into the elite club of compiler writers and make you feel accomplished. But remember: these days, compiler writing is more about language processing than building up that classic pipeline of phases, as explained in this video:



Why Should I Care?

Please *do* write a compiler! Writing a compiler will:

1. Make you a better programmer in general, as you will better understand a language's intricacies and obscurities. You will even start making sense of those ridiculous error messages from other people's compilers.

2. Help you write language tools such as preprocessors, linkers, loaders, debuggers, profilers, syntax highlighters, text formatters, code coverage analyzers, database query languages, and IDEs, or your very own command processors, message scanners, and more. Think you won't ever write such things? A lot of people think that at first, but end up doing so.

3. Improve your knowledge of many related technologies: generalized optimization tasks, advanced computer architectures, graphics interfaces, device controllers, virtual machines, and the translation of human languages.

4. Make you a better computer scientist, because compiler technology spans so many areas of the discipline, including:
    - Formal Language Theory
        - Grammars
        - Parsing
        - Computability

- Programming Languages
  - Syntax
  - Semantics (both static and dynamic)
  - Support for Paradigms (OOP, functional, logic, stack-based)
  - Details of parallelism, metaprogramming, etc.
  - Abstract languages and virtual machines
  - Concepts, such as: sequencing, conditional execution, iteration, recursion, functional decomposition, modularity, synchronization, metaprogramming, binding, scope, extent, volatile, const, subtype, pattern matching, type inference, closures, prototypes, introspection, instrumentation, annotations, decorators, memoization, traits, streams, monads, actors, mailboxes, comprehensions, continuations, wildcards, promises, regular expressions, proxies, transactional memory, inheritance, polymorphism, parameter modes, etc.
- Computer Architecture
  - Instruction sets
  - RISC vs. CISC
  - Pipelines
  - Cores
  - Clock cycles, etc.
- Algorithms and Data Structures
  - Regular expressions
  - Parsing algorithms
  - Graph algorithms
  - Union-find
  - Dynamic programming
  - Learning
- Software Engineering (because compilers are generally large and complex)
  - Locality
  - Caching
  - Componentization, APIs, Reuse
  - Synchronization

In other words, compiler writing is almost the perfect practical activity to bring the most fundamental theories of our discipline to life.

5. Give you practice designing and building large systems, as you grapple with choices and weigh tradeoffs such as:
    - Should I make a one-shot translator or make several layered independent "translation passes"?
    - Should it be quick-and-dirty or take its time generating amazing (optimized) code?
    - Should it stop at the first error? When should it stop? Should it...autocorrect?!
    - What parts should I build myself and when should I use tools?
    - Is retargetability important?
    - Should I just write the front-end only (say, targeting LLVM, RTL, or the JVM)? Or maybe just do source-to-source **transpilation** to C or JavaScript? Or go all the way to assembly, or machine code?

6. Make you a better language designer. People might not use your language, no matter how cool, if it can't be implemented efficiently. You'll learn the impact of type systems, preprocessors, garbage collection and the like on both compile-time and run-time performance.

7. Give you cred in the Programming Languages Community. A compiler course is really just a Programming Languages II course. In Programming Languages I, you learn about **specifying** and **using** languages. In Programming Languages II, you learn about **designing** and **implementing** languages. Implementation is interpretation and translation.

8. Give you cred on the street. "Woah, you wrote a *compiler*?!"

Convinced yet?

Some Good Reads

If you're still a little nervous, here are some nice reads that might get you excited to write compilers because (1) they are cool, and (2) they might not be as hard to write as everyone thinks they are:

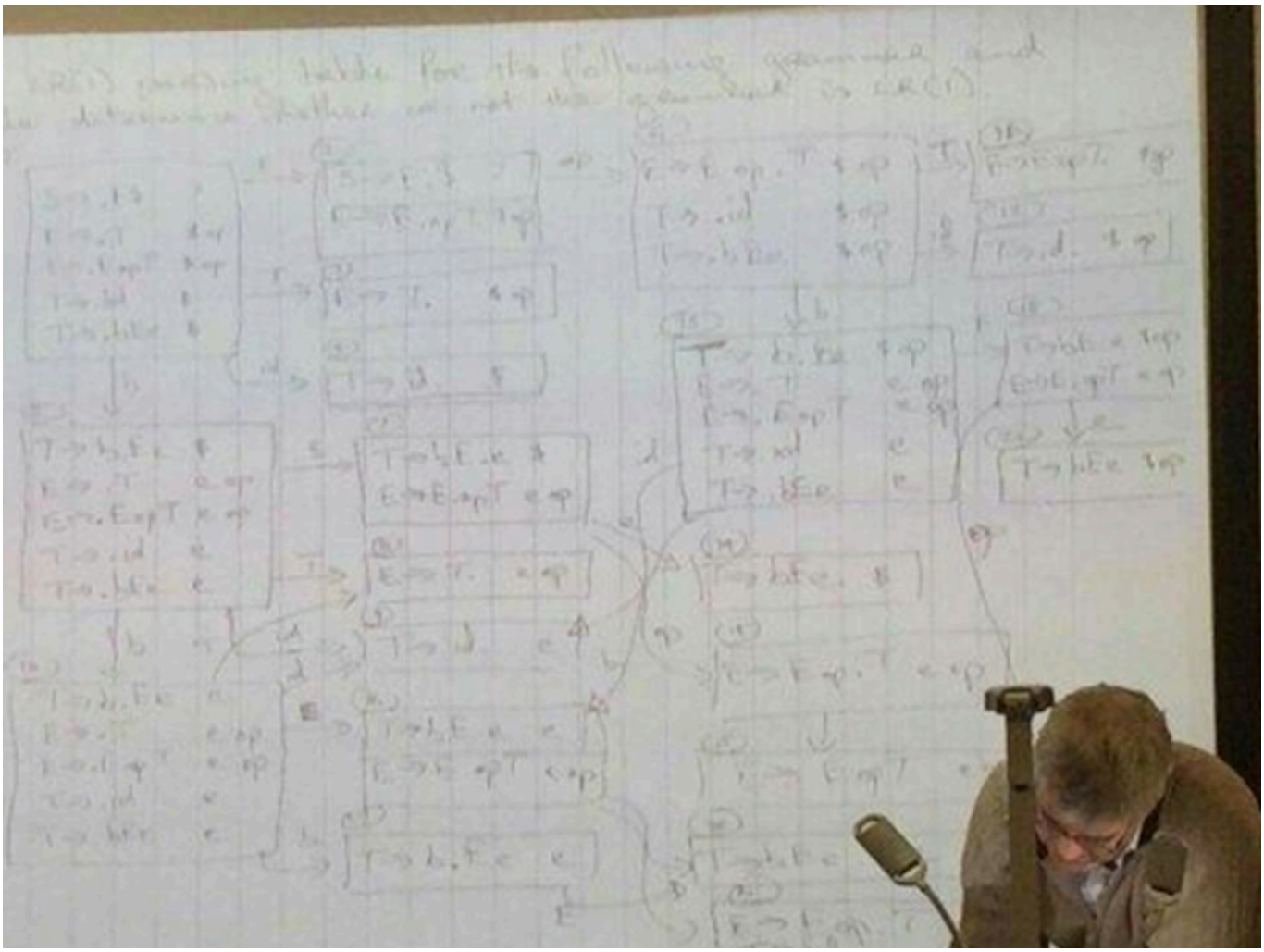
- Want to Write a Compiler? Just Read These Two Papers
- A Forgotten Principle of Compiler Design
- On the Madness of Optimizing Compilers
- Things That Turbo Pascal is Smaller Than

In addition, please browse this nice list of resources and select a few items from it for self-study.

Here's a tutorial.

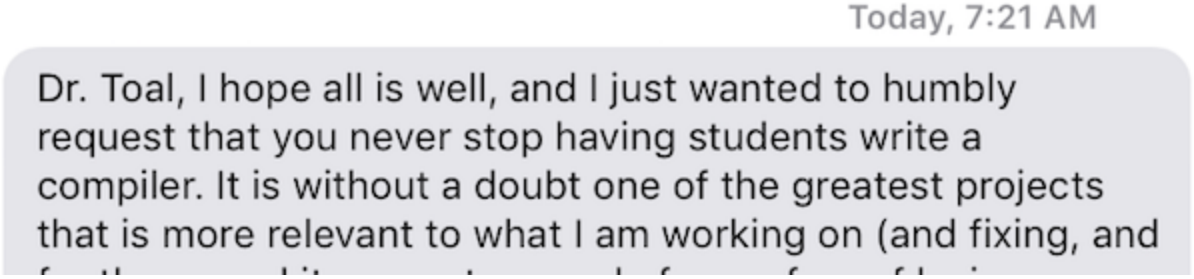
Should I Take A Compiler Class?

Good question. You are probably worried about lectures that look like this (found here):



Or maybe you are worried about following advice like these answers on StackExchange.

Don't worry. It doesn't have to be this way. You will learn by doing. With your friends. Here's a testimonial from a successful alum:



Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

- What is a program?
- What is an interpreter?
- What is a translator?
- What is a compiler?
- Name two kinds of translators other than a compiler.
- What is the difference between AOT and JIT?
- What are informal names for a compiler's front end and its back end?
- Why are translators split into analyzers and generators?
- What happens in the "middle end" of a compiler?
- What happens in lexical analysis?
- What happens during parsing?
- What is the difference between a lexical error and a syntax error?
- What happens during static analysis?
- What is meant by intermediate code generation?
- What are basic blocks?
- What does a control flow graph look like?
- What is the really, really popular middle- and back-end compiler infrastructure called?
- Who was the initial creator of LLVM?
- Why might one use LLVM as a compiler target instead of just targeting C?
- How has compiler technology changed since the first edition of Dragon Book came out?
- Name five reasons why writing a compiler is likely to positively impact your career.
- What has traditionally scared people away from taking compiler construction courses?

Summary

We've covered:

- ✅ What a program is, what an interpreter is, and what a compiler is
- ✅ The difference between AOT and JIT compilation
- ✅ Why translation has analysis and generation phases
- ✅ A simplified architecture of a classic compiler
- ✅ Real compilers often use pre-existing solutions for their middle and back ends
- ✅ Places you can go to study and experiment with real compilers
- ✅ Many good reasons to learn about compilers and write your own

How to Write a Compiler

A step-by-step guide is sometimes just what you need.

CONTENTS

About This Guide • Understand Your Language! • GitHub Setup • The Project Skeleton • Stub Out The Main Files • Begin the Parser, Tests First • Write the Parser • Design the Program Representation • Build and Analyze the Representation • Optimization • Generate Target Code • Now we can finish the compiler test • Thanks!

About This Guide

Want to write a compiler? These notes go over the way that I personally like to write compilers, so you'll find this tutorial very opinionated.

We'll be discussing compiler writing by using a specific example for a language called Carlos. You can use the guide as a code along and create your own Carlos compiler, or you can use the guide to create a compiler for your own language, substituting your language name, syntax, features, etc. for anything in this guide that is Carlos-specific.

This guide assumes you have a bit of programming experience, so before you start, make sure you have:

- ✅ The latest versions of **node**, **npm**, and **git** installed
- ✅ Command line skills
- ✅ GitHub experience

Understand Your Language!

It doesn't matter whether you are designing a language yourself or implementing a complier for an existing language, you need to deeply understand the language. You must, in either case:

- ✅ Understand the intended audience
- ✅ Understand the kinds of applications the language is best suited for
- ✅ Understand the syntax and semantics deeply
- ✅ Write a bunch of example programs

If you are designing the language yourself:

- ✅ Use an interactive language design tool to help you craft a syntax

- ✅ Write a language specification

For this guide, I've already created and wrote a specification for Carlos. To best follow along:

- ✅ Read the Carlos language specification
- ✅ Try your hand at writing example Carlos programs
- ✅ Seriously, read the Carlos language specification

If you are building your own language, customize the steps above accordingly.

> ***Language Design is Iterative***
>
> *You're unlikely to get the design perfectly right at first. Writing the compiler will expose some design flaws or implementation difficulties that you did not foresee. Don't feel bad about changing the language design (perhaps several times) as you implement.*

GitHub Setup

GitHub is a great place to do your work! To get started:

- ✅ At github.com, perform the *New Repository* action
- ✅ Enter a project name and description
- ✅ Leave the *Public* radio button selected
- ✅ Make sure to check *Add a README file*
- ✅ Add the *.gitignore* file with the *Node* template
- ✅ Choose the MIT license
- ✅ Click the *Create repository* button

The Project Skeleton

Go to your terminal app and:

- ✅ Change to the directory where you keep your projects
- ✅ `git clone` the project you just created
- ✅ Change to the project root folder

You should have these files:

```
.
├── .gitignore
├── LICENSE
└── README.md
```

Let's start jamming!

- ✅ Open the project in your favorite editor
- ✅ Make sure your editor has the Prettier extension installed and that it is set to format on save
- ✅ Create a *docs* folder and add a logo to it
- ✅ In the *README.md* add a link to your logo, write the language name, and add text about your language (who it's for, some of its features, some example programs, and any other notes about the project that might be helpful to readers)
- ✅ Optionally, customize Prettier settings in a new *.prettierrc.json* file
- ✅ Create an *examples* folder and add a bunch of example programs

The project tree should now look something like this:

```
.
├── .gitignore
├── LICENSE
├── README.md
├── .prettierrc.json
├── docs
│   └── carlos-logo.png
└── examples
    ├── hello.carlos
    ├── fizzbuzz.carlos
    └── gcd.carlos
```

We've done a lot, so:

- ✅ `git add .`
- ✅ `git commit -m 'Basic project setup'`
- ✅ `git push`

We'll be using Node.js, so let's perform the standard Node project setup with `npm init` and bring in third-party modules. I like the Ohm language processing library, c8 for test coverage, and a pretty little module for visualizing program representations called graph-stringify:

- ✅ Run `npm init -y`
- ✅ Run `npm install ohm-js graph-stringify`
- ✅ Run `npm install c8 --save-dev`

This creates *package.json*, *package-lock.json*, and *node_modules*. Next, you will need to make some hand-edits to *package.json*:

- ✅ Update the `test` command to: `c8 node --test test/**/*.js`
- ✅ Add to the top-level the property `"type": "module"`
- ✅ Make any other changes or additions you like, such as adding to the author and description properties.

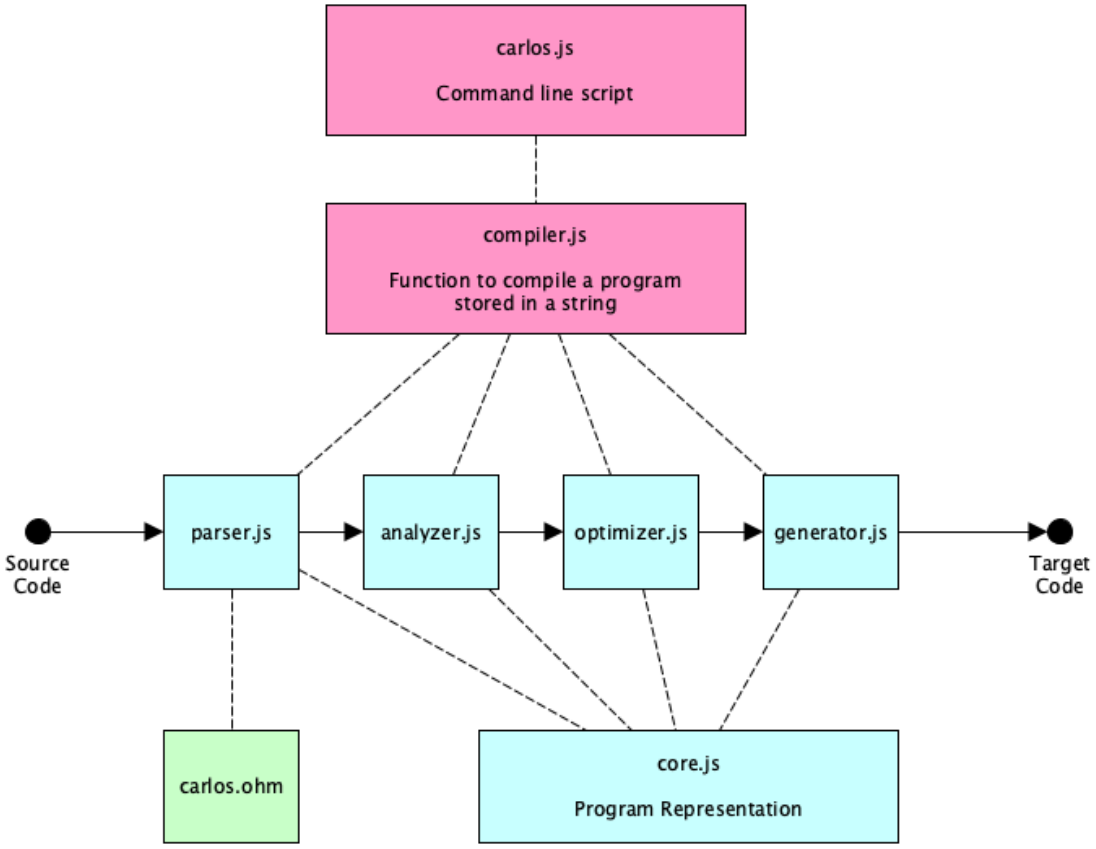
Here's what my *package.json* looks like; yours will be a bit different:

package.json

```
{
  "name": "carlos",
  "version": "2.0.0",
  "description": "A compiler for the Carlos programming language",
  "main": "carlos.js",
  "type": "module",
```

```
  "directories": {
    "doc": "docs",
    "example": "examples",
    "test": "test"
  },
  "scripts": {
    "test": "c8 node --test test/**/*.test.js"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/rtoal/carlos-lang.git"
  },
  "keywords": [],
  "author": "rtoal",
  "license": "MIT",
  "bugs": {
    "url": "https://github.com/rtoal/carlos-lang/issues"
  },
  "homepage": "https://github.com/rtoal/carlos-lang#readme",
  "dependencies": {
    "graph-stringify": "^1.1.0",
    "ohm-js": "^17.1.0"
  },
  "devDependencies": {
    "c8": "^10.1.3"
  }
}
```

Now let's stub out all the source and test files. I like to use this architecture to start:



Perform the set up now.

✅ Create project files to match the structure below. You can do this on the command line (with `mkdir` and `touch` commands 💪) or use your IDE's buttons. (We're not showing the *node_modules* folder in this file tree for obvious reasons.)

```
.
├── .gitignore
├── LICENSE
├── README.md
├── package.json            -- created with npm init
├── package-lock.json       -- also auto-generated, just leave it alone
├── .prettierrc.json        -- bc we all have our own preferences 🤪
├── docs
│   ├── carlos-logo.png
│   └── ...                 -- eventually will hold a companion website
├── examples
│   └── ...                 -- good to have examples so users can see th
├── src
│   ├── carlos.ohm          -- nice to have the grammar in its own file
│   ├── carlos.js           -- to run from command line
│   ├── compiler.js         -- to allow the compiler to be used as a lib
│   ├── core.js             -- defines the internal program representati
│   ├── parser.js           -- simple parser that uses Ohm
│   ├── analyzer.js         -- builds the program rep and checks it
│   ├── optimizer.js        -- makes it faster, right?
│   └── generator.js        -- backend: translates to the target languag
└── test
    ├── compiler.test.js
    ├── parser.test.js
    ├── analyzer.test.js
    ├── optimizer.test.js
    └── generator.test.js
```

✅ Make sure you've set up testing properly. Stub out *test/compiler.test.js* like so:

```
import assert from "node:assert/strict"
describe("The project setup", () => {
  it("can at least run tests", () => {
    assert.deepEqual(1, 1)
  })
})
```

✅ Run `npm test` and make sure there are no errors. Fix any setup problems you find.
✅ git add, commit, and push

Stub Out The Main Files

Each of the files in the middle row of the diagram will export a single file handling a particular compilation phase. The parser will process the source code to a **match object**; the analyzer will process the match to a **program representation**; the optimizer will **improve** the program representation (hopefully making it faster and smaller); and the generator will process the representation to **target code**.

✅ Stub *parser.js* as follows:

```
export default function parse(sourceCode) {
  throw new Error("Not implemented")
}
```

✅ Stub *analyzer.js* as follows:

```
export default function analyze(match) {
  throw new Error("Not implemented")
}
```

✅ Stub *optimizer.js* as follows:

```
export default function optimize(program) {
  throw new Error("Not implemented")
}
```

✅ Stub *generator.js* as follows:

```
export default function generate(program) {
  throw new Error("Not implemented")
}
```

We're going to build the compiler so that we can see the output of the compiler at each step. This will help us while we are developing it, and it might even be convenient for our users. So when running from the command

line, we'd like to have these options:

`node src/carlos.js examples/gcd.carlos parsed`

Prints the result of parsing the program.

`node src/carlos.js examples/gcd.carlos analyzed`

Prints the result of analyzing the program, namely the internal representation, in as decent a form as we can make it.

`node src/carlos.js examples/gcd.carlos optimized`

Prints the result of optimizing the program.

`node src/carlos.js examples/gcd.carlos js`

Prints the result of generating the program, namely the target program.

Let's separate the user interface of the command line app from a function that runs the compiler programatically. This not only demonstrates the good engineering practice known as Separation of Concerns, but also makes the compiler easier to test.

The command line script will do some checking of the command line arguments and take care of reporting. It will ensure that different outputs are rendered nicely, and that error messages appear in red:

Here's the command line script:

carlos.js

```
#! /usr/bin/env node

import * as fs from "node:fs/promises"
import stringify from "graph-stringify"
import compile from "./compiler.js"

const help = `Carlos compiler

Syntax: carlos <filename> <outputType>

Prints to stdout according to <outputType>, which must be one of:

  parsed     a message that the program was matched ok by the grammar
  analyzed   the statically analyzed representation
  optimized  the optimized semantically analyzed representation
  js         the translation to JavaScript
`

async function compileFromFile(filename, outputType) {
  try {
    const buffer = await fs.readFile(filename)
    const compiled = compile(buffer.toString(), outputType)
    console.log(stringify(compiled, "kind") || compiled)
  } catch (e) {
    console.error(`\u001b[31m${e}\u001b[39m`)
    process.exitCode = 1
```

```
  }
}

if (process.argv.length !== 4) {
  console.log(help)
} else {
  compileFromFile(process.argv[2], process.argv[3])
}
```

The *compiler.js* file does the actual compiling:

compiler.js

```
import parse from "./parser.js"
import analyze from "./analyzer.js"
import optimize from "./optimizer.js"
import generate from "./generator.js"

export default function compile(source, outputType) {
  if (!["parsed", "analyzed", "optimized", "js"].includes(outputType)) {
    throw new Error("Unknown output type")
  }
  const match = parse(source)
  if (outputType === "parsed") return "Syntax is ok"
  const analyzed = analyze(match)
  if (outputType === "analyzed") return analyzed
  const optimized = optimize(analyzed)
  if (outputType === "optimized") return optimized
  return generate(optimized)
}
```

Let's create these files.

- ✅ Create *src/carlos.js* (or, the equivalent for your own language).
- ✅ Create *src/compiler.js*.
- ✅ git add, commit, and push.

Begin the Parser, Tests First

We want to write tests FIRST! Compilers need lots and lots of tests. The first part of the compiler we will work on is the **parser** portion, which will attempt to match a source code file against that grammar and return a **match result**, or throw an error if a match was not possible—that is, the parser found lexical or syntax errors in the soruce program.

We have to test both (1) that Carlos programs we expect to be syntactically correct are indeed correct, and (2) that Carlos programs with syntax errors should have those errors detected at the point we expect them to be

detected.Here's how the parser test should look:

parser.test.js

```
import { describe, it } from "node:test"
import assert from "node:assert/strict"
import parse from "../src/parser.js"

// Programs expected to be syntactically correct
const syntaxChecks = [
  ["simplest syntactically correct program", "break;"],
  ["multiple statements", "print(1);\nbreak;\nx=5; return; return;"],
  ["variable declarations", "let e=99*1;\nconst z=false;"],
  ["type declarations", "struct S {x:T1 y:T2 z:bool}"],
  ["function with no params, no return type", "function f() {}"],
  ["function with one param", "function f(x: int) {}"],
  ["function with two params", "function f(x: int, y: boolean) {}"],
  ["function with no params + return type", "function f(): int {}"],
  ["function types in params", "function f(g: (int)->boolean) {}"],
  ["function types returned", "function f(): (int)->(int)->void {}"],
  ["array type for param", "function f(x: [[[boolean]]]) {}"],
  ["array type returned", "function f(): [[int]] {}"],
  ["optional types", "function f(c: int?): float {}"],
  ["assignments", "a--; c++; abc=9*3; a=1;"],
  ["complex var assignment", "c(5)[2] = 100;c.p.r=1;c.q(8)[2](1,1).z=1;"],
  ["complex var bumps", "c(5)[2]++;c.p.r++;c.q(8)[2](1,1).z--;"],
  ["call in statement", "let x = 1;\nf(100);\nprint(1);"],
  ["call in exp", "print(5 * f(x, y, 2 * y));"],
  ["short if", "if true { print(1); }"],
  ["longer if", "if true { print(1); } else { print(1); }"],
  ["even longer if", "if true { print(1); } else if false { print(1);}"],
  ["while with empty block", "while true {}"],
  ["while with one statement block", "while true { let x = 1; }"],
  ["repeat with long block", "repeat 2 { print(1);\nprint(2);print(3); }"],
  ["if inside loop", "repeat 3 { if true { print(1); } }"],
  ["for closed range", "for i in 2...9*1 {}"],
  ["for half-open range", "for i in 2..<9*1 {}"],
  ["for collection-as-id", "for i in things {}"],
  ["for collection-as-lit", "for i in [3,5,8] {}"],
  ["conditional", "return x?y:z?y:p;"],
  ["??", "return a ?? b ?? c ?? d;"],
  ["ors can be chained", "print(1 || 2 || 3 || 4 || 5);"],
  ["ands can be chained", "print(1 && 2 && 3 && 4 && 5);"],
  ["bitwise ops", "return (1|2|3) + (4^5^6) + (7&8&9);"],
  ["relational operators", "print(1<2||1<=2||1==2||1!=2||1>=2||1>2);"],
  ["shifts", "return 3 << 5 >> 8 << 13 >> 21;"],
  ["arithmetic", "return 2 * x + 3 / 5 - -1 % 7 ** 3 ** 3;"],
  ["length", "return #c; return #[1,2,3];"],
  ["boolean literals", "let x = false || true;"],
```

```
  ["all numeric literal forms", "print(8 * 89.123 * 1.3E5 * 1.3E+5 * 1.3E-5);"],
  ["empty array literal", "print([int]());"],
  ["nonempty array literal", "print([1, 2, 3]);"],
  ["some operator", "return some dog;"],
  ["no operator", "return no dog;"],
  ["random operator", "return random [1, 2, 3];"],
  ["parentheses", "print(83 * (((((((-(13 / 21))))))))) + 1 - 0);"],
  ["variables in expression", "return r.p(3,1)[9]?.x?.y.z.p()(5)[1];"],
  ["more variables", "return c(3).p?.oh(9)[2][2].nope(1)[3](2);"],
  ["indexing array literals", "print([1,2,3][1]);"],
  ["member expression on string literal", `print("hello".append("there"));`],
  ["non-Latin letters in identifiers", "let コンパイラ = 100;"],
  ["a simple string literal", 'print("hello😉😐💀🧕—`");'],
  ["string literal with escapes", 'return "a\\n\\tbc\\\\de\\"fg";'],
  ["u-escape", 'print("\\u{a}\\u{2c}\\u{1e5}\\u{ae89}\\u{1f4a9}\\u{10ffe8}");'],
  ["end of program inside comment", "print(0); // yay"],
  ["comments with no text", "print(1);//\nprint(0);//"],
]

// Programs with syntax errors that the parser will detect
const syntaxErrors = [
  ["non-letter in an identifier", "let ab😭c = 2;", /Line 1, col 7:/],
  ["malformed number", "let x= 2.;", /Line 1, col 10:/],
  ["a float with an E but no exponent", "let x = 5E * 11;", /Line 1, col 10:/],
  ["a missing right operand", "print(5 -);", /Line 1, col 10:/],
  ["a non-operator", "print(7 * ((2 _ 3));", /Line 1, col 15:/],
  ["an expression starting with a )", "return );", /Line 1, col 8:/],
  ["a statement starting with expression", "x * 5;", /Line 1, col 3:/],
  ["an illegal statement on line 2", "print(5);\nx * 5;", /Line 2, col 3:/],
  ["a statement starting with a )", "print(5);\n)", /Line 2, col 1:/],
  ["an expression starting with a *", "let x = * 71;", /Line 1, col 9:/],
  ["negation before exponentiation", "print(-2**2);", /Line 1, col 10:/],
  ["mixing ands and ors", "print(1 && 2 || 3);", /Line 1, col 15:/],
  ["mixing ors and ands", "print(1 || 2 && 3);", /Line 1, col 15:/],
  ["associating relational operators", "print(1 < 2 < 3);", /Line 1, col 13:/],
  ["while without braces", "while true\nprint(1);", /Line 2, col 1/],
  ["if without braces", "if x < 3\nprint(1);", /Line 2, col 1/],
  ["while as identifier", "let for = 3;", /Line 1, col 5/],
  ["if as identifier", "let if = 8;", /Line 1, col 5/],
  ["unbalanced brackets", "function f(): int[;", /Line 1, col 18/],
  ["empty array without type", "print([]);", /Line 1, col 8/],
  ["random used like a function", "print(random(1,2));", /Line 1, col 15/],
  ["bad array literal", "print([1,2,]);", /Line 1, col 12/],
  ["empty subscript", "print(a[]);", /Line 1, col 9/],
  ["true is not assignable", "true = 1;", /Line 1, col 5/],
  ["false is not assignable", "false = 1;", /Line 1, col 6/],
  ["numbers cannot be subscripted", "print(500[x]);", /Line 1, col 10/],
  ["numbers cannot be called", "print(500(x));", /Line 1, col 10/],
```

```
    ["numbers cannot be dereferenced", "print(500 .x);", /Line 1, col 11/],
    ["no-paren function type", "function f(g:int->int) {}", /Line 1, col 17/],
    ["string lit with unknown escape", 'print("ab\\zcdef");', /col 11/],
    ["string lit with newline", 'print("ab\\zcdef");', /col 11/],
    ["string lit with quote", 'print("ab\\zcdef");', /col 11/],
    ["string lit with code point too long", 'print("\\u{1111111}");', /col 17/],
  ]

  describe("The parser", () => {
    for (const [scenario, source] of syntaxChecks) {
      it(`matches ${scenario}`, () => {
        assert(parse(source).succeeded())
      })
    }
    for (const [scenario, source, errorMessagePattern] of syntaxErrors) {
      it(`throws on ${scenario}`, () => {
        assert.throws(() => parse(source), errorMessagePattern)
      })
    }
  })
```

Let' get to work:

- ✅ Fill in your *parser.test.js* file, using mine as a guide.
- ✅ From the project root folder, run `npm test`

You will see a big scary output that somewhere inside of it all says The requested module '../src/parser.js' does not provide an export named 'default', which just means we haven't actually written the parser yet. This is normal in Test-Driven Development. One nice benefit of TDD is that we have, in writing the test file, spent a lot of time writing Carlos code that should and should not compile, so we are deepening our understanding of the language we are writing a compiler for. And that is good.

Write the Parser

Writing hand-crafted syntax analyzers (parsers) is good, but getting Ohm to generate the parser is better! (I told you these notes were opinionated.) When designing your own language, use the Ohm Editor.

> ***Use the Ohm Editor for Syntax Design***
>
> *THE OHM EDITOR IS AMAZING! You can write a few tests in the Ohm editor and the tests run as you type! Such immediate feedback is crucial in the creative design process.*

Here's the Carlos grammar:

carlos.ohm

```
Carlos {
  Program     = Statement+

  Statement   = VarDecl
              | TypeDecl
              | FunDecl
              | Exp9 ("++" | "--") ";"                        --bump
              | Exp9 "=" Exp ";"                              --assign
              | Exp9_call ";"                                 --call
              | break ";"                                     --break
              | return Exp ";"                                --return
              | return ";"                                    --shortreturn
              | IfStmt
              | LoopStmt

  VarDecl     = (let | const) id "=" Exp ";"
  TypeDecl    = struct id "{" Field* "}"
  Field       = id ":" Type
  FunDecl     = function id Params (":" Type)? Block
  Params      = "(" ListOf<Param, ","> ")"
  Param       = id ":" Type

  Type        = Type "?"                                      --optional
              | "[" Type "]"                                  --array
              | "(" ListOf<Type, ","> ")" "->" Type           --function
              | id                                            --id

  IfStmt      = if Exp Block else Block                       --long
              | if Exp Block else IfStmt                      --elsif
              | if Exp Block                                  --short
  LoopStmt    = while Exp Block                               --while
              | repeat Exp Block                              --repeat
              | for id in Exp ("..." | "..<") Exp Block       --range
              | for id in Exp Block                           --collection
  Block       = "{" Statement* "}"

  Exp         = Exp1 "?" Exp1 ":" Exp                         --conditional
              | Exp1
  Exp1        = Exp1 "??" Exp2                                --unwrapelse
              | Exp2
  Exp2        = Exp3 ("||" Exp3)+                             --or
              | Exp3 ("&&" Exp3)+                             --and
              | Exp3
  Exp3        = Exp4 ("|" Exp4)+                              --bitor
              | Exp4 ("^" Exp4)+                              --bitxor
              | Exp4 ("&" Exp4)+                              --bitand
              | Exp4
```

```
Exp4        = Exp5 ("<="|"<"|"=="|"!="|">="|">") Exp5       --compare
            | Exp5
Exp5        = Exp5 ("<<" | ">>") Exp6                       --shift
            | Exp6
Exp6        = Exp6 ("+" | "-") Exp7                         --add
            | Exp7
Exp7        = Exp7 ("*"| "/" | "%") Exp8                    --multiply
            | Exp8
Exp8        = Exp9 "**" Exp8                                --power
            | Exp9
            | ("#" | "-" | "!" | some | random) Exp9        --unary
Exp9        = true ~mut
            | false ~mut
            | floatlit ~mut
            | intlit ~mut
            | no Type ~mut                                  --emptyopt
            | Exp9 ("(" | "?(") ListOf<Exp, ","> ")" ~mut   --call
            | Exp9 ("[" | "?[") Exp "]"                     --subscript
            | Exp9 ("." | "?.") id                          --member
            | stringlit ~mut
            | id                                            --id
            | Type_array "(" ")" ~mut                       --emptyarray
            | "[" NonemptyListOf<Exp, ","> "]" ~mut         --arrayexp
            | "(" Exp ")" ~mut                              --parens

intlit      = digit+
floatlit    = digit+ "." digit+ (("E" | "e") ("+" | "-")? digit+)?
stringlit   = "\"" char* "\""
char        = ~control ~"\\" ~"\"" any
            | "\\" ("n" | "t" | "\"" | "\\")                --escape
            | "\\u{" hex hex? hex? hex? hex? hex? "}"        --codepoint
control     = "\x00".."\x1f" | "\x80".."\x9f"
hex         = hexDigit
mut         = ~"==" "=" | "++" | "--"

let         = "let" ~alnum
const       = "const" ~alnum
struct      = "struct" ~alnum
function    = "function" ~alnum
if          = "if" ~alnum
else        = "else" ~alnum
while       = "while" ~alnum
repeat      = "repeat" ~alnum
for         = "for" ~alnum
in          = "in" ~alnum
random      = "random" ~alnum
break       = "break" ~alnum
return      = "return" ~alnum
```

```
  some        = "some" ~alnum
  no          = "no" ~alnum
  true        = "true" ~alnum
  false       = "false" ~alnum
  keyword     = let | const | struct | function | if | else | while | repeat
              | for | in | break | return | some | no | random | true | false
  id          = ~keyword letter alnum*

  space      += "//" (~"\n" any)*                          --comment
}
```

The parser itself is surprisingly small! Ohm does a lot of work for you. The parser module needs to first have Ohm create a grammar object from the grammar in the *carlos.ohm* file. Then the `parse` function itself invokes the grammar's match operation to do all the parsing for us:

parser.js

```
// The parse() function uses Ohm to produce a match object for a given
// source code program, using the grammar in the file carlos.ohm.

import * as fs from "node:fs"
import * as ohm from "ohm-js"

const grammar = ohm.grammar(fs.readFileSync("src/carlos.ohm"))

// Returns the Ohm match if successful, otherwise throws an error
export default function parse(sourceCode) {
  const match = grammar.match(sourceCode)
  if (!match.succeeded()) throw new Error(match.message)
  return match
}
```

For our project, we do the following:

- ✅ Fill in *src/carlos.ohm* (or the equivalent if you are working on your own).
- ✅ Fill in *src/parser.js*.
- ✅ Now run those tests (`npm test`), fix your code, run tests some more, fix your code some more, until all the tests pass.
- ✅ Enjoy the green text that says all your tests pass and pat yourself on the back for 100% test coverage.
- ✅ git add, commit, and push.
- ✅ It's a good time to try a manual test to see if the parser can be fired off from the command line. Run one of the examples from the examples folder (if you haven't made one, do so now!) as follows:

```
$ node src/carlos.js examples/gcd.carlos parsed
Syntax is ok
```

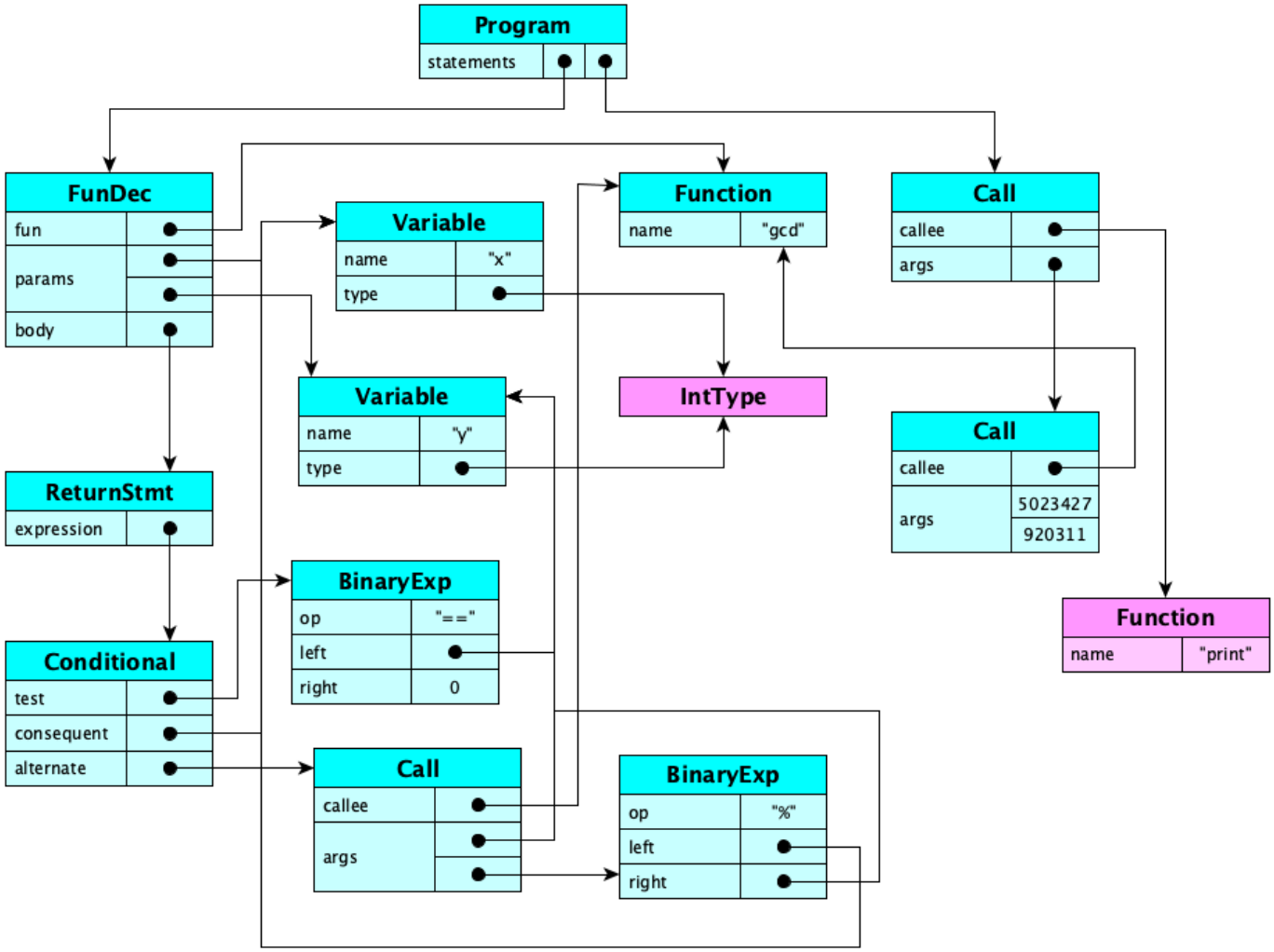
Design the Program Representation

Our parser checks syntax and creates a match result object containing a parse tree. Parse trees are important but more complex than we need for later stages of compilation. We need to build a program representation that makes it easier to do semantic analysis (type checking, etc.)

Let's try to get an idea for what such a representation will look like. Let's start with a sample Carlos program, such as:

```
function gcd(x: int, y: int): int {
  return y == 0 ? x : gcd(y, x % y);
}
print(gcd(5023427, 920311));
```

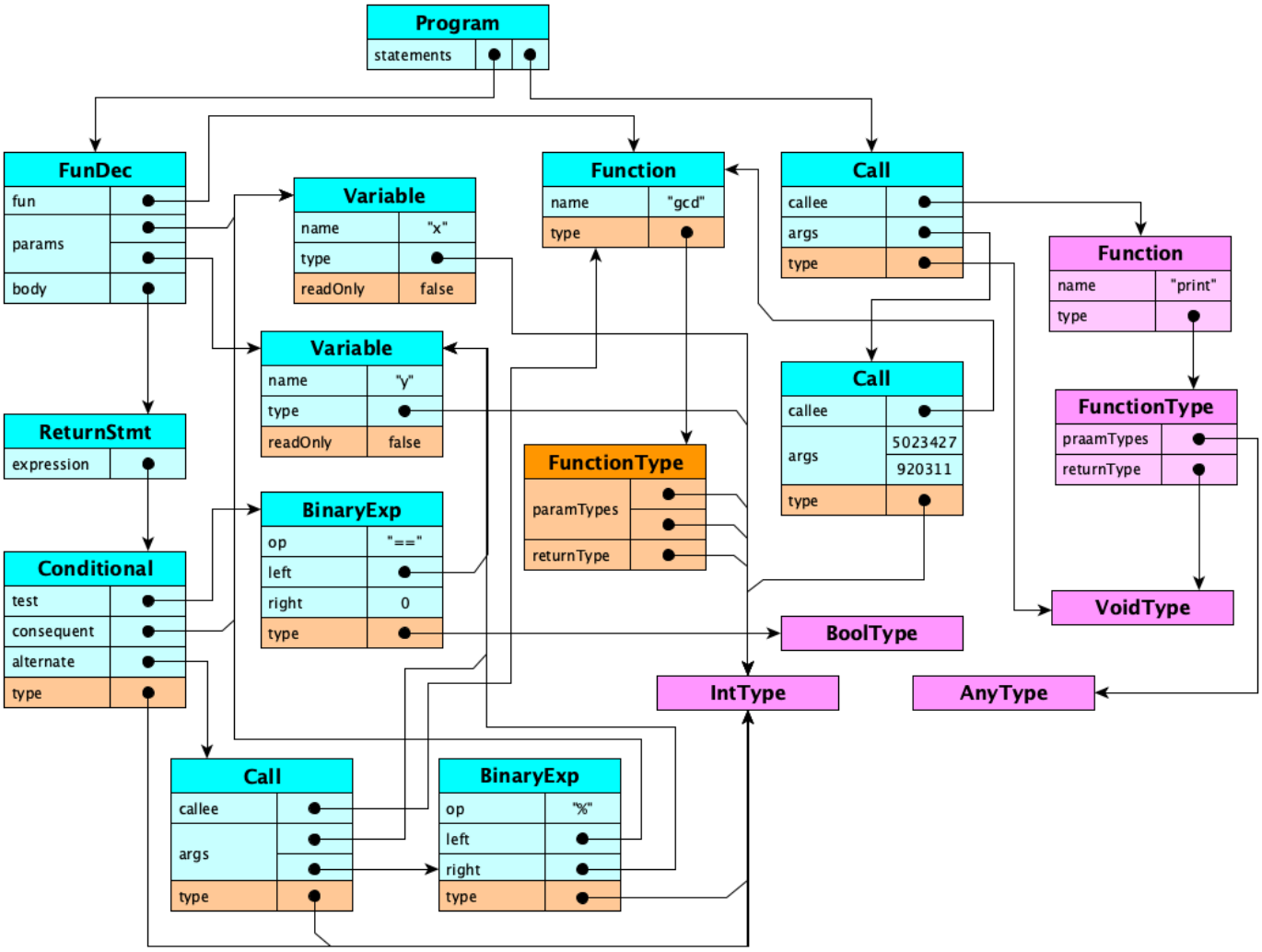
Next, we sketch out *a first pass* of the representation we want, based on our sense of the abstract syntax of the language:



Here we used blue for the nodes in the program we wrote ourselves, and purple for the nodes that come from the Carlos standard library.

Thinking ahead, it's clear we are missing some things. There are some language rules a grammar cannot capture. We have to be able to type check of all our expressions. This includes not only checking the types of operands for

our basic operators, but making sure the types of functions are consistent with their argument types in calls. Calls need to have the right number of arguments, and the type of each argument must match up with the types of the corresponding parameters. Therefore when building the program representation, we need to add types for each of the expressions and functions, too. Also, Carlos has the notion of variables being either mutable or immutable, so all variables, including parameters, should have this property as well:



We've had to bring in more from the standard library, of course. The diagram might look a little scary, but you can get used to it.

Now we're ready to determine all of the node types we will need. To do this, we go through the Ohm grammar and ask ourselves which things are relevant for semantics and which are not. This takes experience, so don't fret if you don't get things perfect at first. This is what I selected for Carlos:

Kind	Properties	Notes
Program	statements	A program contains a list of statements.
VariableDeclaration	variable, initializer	A variable declaration will create a VariableDeclaration node and also a node for the actual variable being declared.
Variable	name, readOnly	Created during variable declarations, parameter declarations, and also when processing for-loops, since the iterator of a for-loop is indeed a variable.
TypeDeclaration	type	A type declaration builds a TypeDeclaration node and also a node for the type being declared. Carlos only allows the declaration of

		new struct types. (Arrays, functions, and optionals are never named; they are built on-the-fly. Nodes for the predefined types int, float, boolean, string, and void will be built automatically at startup.)
IntType		The type of integer values.
FloatType		The type of float values.
BoolType		The type of boolean values.
StringType		The type of string values.
AnyType		The top type (used in Carlos for arguments to the standard `print` function).
VoidType		The bottom type (used in Carlos for the return type of functions that don't return anything).
ArrayType	baseType	An array type.
StructType	name, fields	A struct type.
OptionalType	baseType	An optional type.
Field	name, type	A field in a struct type.
FunctionDeclaration	fun, params, body	A function declaration builds a FunctionDeclaration node and also a node for the actual function being declared.
Function	name, type	The function object built during a function declaration. (If Carlos had anonymous functions, one would be built for those too.)
FunctionType	parameterTypes, returnType	The type that functions have, consisting of the parameter types (in order) and the return type. The returnType can be `void`. There's no good reason for a parameter to have type `void`, though it's not prohibited by the syntax.
Increment	variable	`e++;`.
Decrement	variable	`e--;`.
Assignment	target, source	`v = e;`.
BreakStatement		`break;`
ReturnStatement	expression	A return statement with an expression.
ShortReturnStatement		A return statement without an expression.
IfStatement	test, consequent, alternate	Standard if-then-else statement. The else part can be a statement list or another if statement.
ShortIfStatement	test, consequent	If statement without an else part.
WhileStatement	test, body	Statements of the form `while test {...}`.
RepeatStatement	count, body	Statements of the form `repeat count {...}`.
ForRangeStatement	iterator, low, op, high, body	For loop with low and high values of a range; op is either `..<` or `...`.
ForStatement	iteration, collection, body	Statement for iterating over the elements of an array.

Conditional	test, consequent, alternate	Conditional expression. The only ternary operator in Carlos.
BinaryExpression	op, left, right	An expression made from any of the binary operators.
UnaryExpression	op, operand	An expression made from any of the unary operators.
EmptyOptional	baseType	Represents expressions such as `no string`. Carlos does not have a free-standing `nil` value; the type that the optional wraps *must* be made explicit.
SubscriptExpression	array, index	`a[e]`.
ArrayExpression	elements	`[x, y, z]`.
EmptyArray	baseType	`[](of int)`. Note that Carlos, unlike many other languages, requires you to specify the type for empty arrays.
MemberExpression	object, field	`p.x`.
Call	callee, args	`f(x, y)`.

We'll be storing a `type` field in all expressions node, too. While we probably could compute these on the fly during type checking, it's faster to compute them up front and store them in the object.

For simplicity, we're going to handle:

- Carlos float literals directly as JavaScript numbers
- Carlos integer literals directly as JavaScript BigInts
- Carlos string literals directly as JavaScript strings
- Carlos identifiers directly as JavaScript symbols

We will represent these nodes as plain old objects. In *src/core.js* we'll include functions to build these objects, together withe objects for all of the predefined identifiers that are part of the standard library:

core.js

```
export function program(statements) {
  return { kind: "Program", statements }
}

export function variableDeclaration(variable, initializer) {
  return { kind: "VariableDeclaration", variable, initializer }
}

export function variable(name, readOnly, type) {
  return { kind: "Variable", name, readOnly, type }
}

export function typeDeclaration(type) {
  return { kind: "TypeDeclaration", type }
}

export const boolType = { kind: "BoolType" }
```

```
export const intType = { kind: "IntType" }
export const floatType = { kind: "FloatType" }
export const stringType = { kind: "StringType" }
export const voidType = { kind: "VoidType" }
export const anyType = { kind: "AnyType" }

export function structType(name, fields) {
  return { kind: "StructType", name, fields }
}

export function field(name, type) {
  return { kind: "Field", name, type }
}

export function functionDeclaration(fun, params, body) {
  return { kind: "FunctionDeclaration", fun, params, body }
}

export function fun(name, type) {
  return { kind: "Function", name, type }
}

export function arrayType(baseType) {
  return { kind: "ArrayType", baseType }
}

export function functionType(paramTypes, returnType) {
  return { kind: "FunctionType", paramTypes, returnType }
}

export function optionalType(baseType) {
  return { kind: "OptionalType", baseType }
}

export function increment(variable) {
  return { kind: "Increment", variable }
}

export function decrement(variable) {
  return { kind: "Decrement", variable }
}

export function assignment(target, source) {
  return { kind: "Assignment", target, source }
}

export const breakStatement = { kind: "BreakStatement" }
```

```
export function returnStatement(expression) {
  return { kind: "ReturnStatement", expression }
}

export function shortReturnStatement() {
  return { kind: "ShortReturnStatement" }
}

export function ifStatement(test, consequent, alternate) {
  return { kind: "IfStatement", test, consequent, alternate }
}

export function shortIfStatement(test, consequent) {
  return { kind: "ShortIfStatement", test, consequent }
}

export function whileStatement(test, body) {
  return { kind: "WhileStatement", test, body }
}

export function repeatStatement(count, body) {
  return { kind: "RepeatStatement", count, body }
}

export function forRangeStatement(iterator, low, op, high, body) {
  return { kind: "ForRangeStatement", iterator, low, op, high, body }
}

export function forStatement(iterator, collection, body) {
  return { kind: "ForStatement", iterator, collection, body }
}

export function conditional(test, consequent, alternate, type) {
  return { kind: "Conditional", test, consequent, alternate, type }
}

export function binary(op, left, right, type) {
  return { kind: "BinaryExpression", op, left, right, type }
}

export function unary(op, operand, type) {
  return { kind: "UnaryExpression", op, operand, type }
}

export function emptyOptional(baseType) {
  return { kind: "EmptyOptional", baseType, type: optionalType(baseType) }
}
```

```
export function subscript(array, index) {
  return { kind: "SubscriptExpression", array, index, type: array.type.baseType }
}

export function arrayExpression(elements) {
  return { kind: "ArrayExpression", elements, type: arrayType(elements[0].type) }
}

export function emptyArray(type) {
  return { kind: "EmptyArray", type }
}

export function memberExpression(object, op, field) {
  return { kind: "MemberExpression", object, op, field, type: field.type }
}

export function functionCall(callee, args) {
  return { kind: "FunctionCall", callee, args, type: callee.type.returnType }
}

export function constructorCall(callee, args) {
  return { kind: "ConstructorCall", callee, args, type: callee }
}

// These local constants are used to simplify the standard library definitions.
const floatToFloatType = functionType([floatType], floatType)
const floatFloatToFloatType = functionType([floatType, floatType], floatType)
const stringToIntsType = functionType([stringType], arrayType(intType))
const anyToVoidType = functionType([anyType], voidType)

export const standardLibrary = Object.freeze({
  int: intType,
  float: floatType,
  boolean: boolType,
  string: stringType,
  void: voidType,
  any: anyType,
  π: variable("π", true, floatType),
  print: fun("print", anyToVoidType),
  sin: fun("sin", floatToFloatType),
  cos: fun("cos", floatToFloatType),
  exp: fun("exp", floatToFloatType),
  ln: fun("ln", floatToFloatType),
  hypot: fun("hypot", floatFloatToFloatType),
  bytes: fun("bytes", stringToIntsType),
  codepoints: fun("codepoints", stringToIntsType),
})
```

```
// We want every expression to have a type property. But we aren't creating
// special entities for numbers, strings, and booleans; instead, we are
// just using JavaScript values for those. Fortunately we can monkey patch
// the JS classes for these to give us what we want.
String.prototype.type = stringType
Number.prototype.type = floatType
BigInt.prototype.type = intType
Boolean.prototype.type = boolType
```

The graph-stringify module we installed at the beginning of this guide will come in handy for displaying program representations. The Carlos program in this section will be "stringified" to the following:

```
 1 | Program statements=[#2,#14]
 2 | FunctionDeclaration fun=#3 params=[#6,#7] body=[#8]
 3 | Function name='gcd' type=#4
 4 | FunctionType paramTypes=[#5,#5] returnType=#5
 5 | IntType
 6 | Variable name='x' readOnly=false type=#5
 7 | Variable name='y' readOnly=false type=#5
 8 | ReturnStatement expression=#9
 9 | Conditional test=#10 consequent=#6 alternate=#12 type=#5
10 | BinaryExpression op='==' left=#7 right=0n type=#11
11 | BoolType
12 | FunctionCall callee=#3 args=[#7,#13] type=#5
13 | BinaryExpression op='%' left=#6 right=#7 type=#5
14 | FunctionCall callee=#15 args=[#19] type=#18
15 | Function name='print' type=#16
16 | FunctionType paramTypes=[#17] returnType=#18
17 | AnyType
18 | VoidType
19 | FunctionCall callee=#3 args=[5023427n,920311n] type=#5
```

We'll see this in the next section.

At this point:

- ✅ Fill in *src/core.js*.
- ✅ Run `npm test` and make sure nothing broke.
- ✅ git add, commit, and push.

Build and Analyze the Representation

While it would feel nice to first build a program representation and then analyze it (separation of concerns again!), it's kind of hard to untangle these phases and they are best done together. After all, you sometimes need to have made certain semantic checks and have performed type inference in order to even build the representation. So in our analyzer module, our `analyze` function will accept the match from the parser and give back a program representation that it both builds and analyzes.

Understanding Semantic Analysis

As you know, it's very possible that your program has no syntax errors, but is still semantically meaningless. In a static language like Carlos, the semantic checking phase of the compiler is huge! The following are just *some* of the rules that must be enforced:

- Identifiers must be in scope before they are used. (Identifiers for the built-in types `int` and `string`, and all the built in functions, are assumed to be pre-declared in a scope that encloses the entire program.)
- Identifiers must not be declared more than once in a scope.
- In an assignment, the type of the source must be compatible with the type of the target.
- Read-only variables must not be assigned to.
- In a call (or operator invocation), the number of arguments must match the number of parameters in the callee.
- In a call (or operator invocation), the types of the arguments must be compatible with the types of the corresponding parameters.
- The order, names, and types of bindings in a struct expression must match the fields of the corresponding struct type exactly. By match we mean the order and names are exactly the same, and the types of the struct expression arguments must be compatible with the declared types of the fields.
- The types of the arms of a conditional operator expression must be the same.
- In the expression x `??` y, x must be of an optional type and y must be of the wrapped type.
- In an array expression $[e_1, ..., e_2]$, all elements e_i must have the same type.
- In a subscript expression `a[e]`, a must have an array type and e must have integer type.
- In a member expression `x.y`, x must have a struct type and y must be a field of that type.
- Break statements can only appear in loops.
- Return statements can only appear in functions.
- The type of a return expression must match the return type of the function
- A `break` expression may only appear within a `for` or `while` expression; however, watch out for function declarations inside of loops—see why?

There are many more! The full list of checks are determined by reading the language spec, and they become clear as you write the compiler, and importantly, write all of its tests.

Semantic analysis proceeds by traversing the parse tree returned by the parser and performing **actions** at each node. These actions include both (1) the construction of program representation nodes AND the validation of the contextual, semantic rules of the language that cannot be captured by the grammar. These contextual rules are enforced with the aid of a **context** object. The context holds all the local declarations we've seen in this scope (for lookup later), whether we are in a loop (so we can check the legality of `break` statements), the function we are processing, in any, (so we can check the legality of `return` statements), and a reference to the *enclosing context*, so we can search for non-local identifier declarations. During analysis we not only perform legality checks, but also create entities from declarations and resolve identifiers to the entities they refer to. That's a lot, so let's get back to building the compiler.

Semantic Analysis Tests

We should write tests for programs that we expect to be semantically correct, those that are syntactically correct but have semantic errors, and tests that our program representation is properly rewritten:

analyzer.test.js

```
import { describe, it } from "node:test"
import assert from "node:assert/strict"
import parse from "../src/parser.js"
import analyze from "../src/analyzer.js"
import { program, variableDeclaration, variable, binary, floatType } from "../src/core.js"

// Programs that are semantically correct
const semanticChecks = [
  ["variable declarations", 'const x = 1; let y = "false";'],
  ["complex array types", "function f(x: [[[int?]]?]) {}"],
  ["increment and decrement", "let x = 10; x--; x++;"],
  ["initialize with empty array", "let a = [int]();"],
  ["type declaration", "struct S {f: (int)->boolean? g: string}"],
  ["assign arrays", "let a = [int]();let b=[1];a=b;b=a;"],
  ["assign to array element", "const a = [1,2,3]; a[1]=100;"],
  ["initialize with empty optional", "let a = no int;"],
  ["short return", "function f() { return; }"],
  ["long return", "function f(): boolean { return true; }"],
  ["assign optionals", "let a = no int;let b=some 1;a=b;b=a;"],
  ["return in nested if", "function f() {if true {return;}}"],
  ["break in nested if", "while false {if true {break;}}"],
  ["long if", "if true {print(1);} else {print(3);}"],
  ["elsif", "if true {print(1);} else if true {print(0);} else {print(3);}"],
  ["for over collection", "for i in [2,3,5] {print(1);}"],
  ["for in range", "for i in 1..<10 {print(0);}"],
  ["repeat", "repeat 3 {let a = 1; print(a);}"],
  ["conditionals with ints", "print(true ? 8 : 5);"],
  ["conditionals with floats", "print(1<2 ? 8.0 : -5.22);"],
  ["conditionals with strings", 'print(1<2 ? "x" : "y");'],
  ["??", "print(some 5 ?? 0);"],
  ["nested ??", "print(some 5 ?? 8 ?? 0);"],
  ["||", "print(true||1<2||false||!true);"],
  ["&&", "print(true&&1<2&&false&&!true);"],
  ["bit ops", "print((1&2)|(9^3));"],
  ["relations", 'print(1<=2 && "x">"y" && 3.5<1.2);'],
  ["ok to == arrays", "print([1]==[5,8]);"],
  ["ok to != arrays", "print([1]!=[5,8]);"],
  ["shifts", "print(1<<3<<5<<8>>2>>0);"],
  ["arithmetic", "let x=1;print(2*3+5**-3/2-5%8);"],
  ["array length", "print(#[1,2,3]);"],
  ["optional types", "let x = no int; x = some 100;"],
  ["random with array literals, ints", "print(random [1,2,3]);"],
  ["random with array literals, strings", 'print(random ["a", "b"]);'],
  ["random on array variables", "let a=[true, false];print(random a);"],
  ["variables", "let x=[[[[1]]]]; print(x[0][0][0][0]+2);"],
  ["pseudo recursive struct", "struct S {z: S?} let x = S(no S);"],
  ["nested structs", "struct T{y:int} struct S{z: T} let x=S(T(1)); print(x.z.y);"],
```

```
  ["member exp", "struct S {x: int} let y = S(1);print(y.x);"],
  ["optional member exp", "struct S {x: int} let y = some S(1);print(y?.x);"],
  ["subscript exp", "let a=[1,2];print(a[0]);"],
  ["array of struct", "struct S{} let x=[S(), S()];"],
  ["struct of arrays and opts", "struct S{x: [int] y: string??}"],
  ["assigned functions", "function f() {}\nlet g = f;g = f;"],
  ["call of assigned functions", "function f(x: int) {}\nlet g=f;g(1);"],
  ["type equivalence of nested arrays", "function f(x: [[int]]) {} print(f([[1],[2]]));"],
  [
    "call of assigned function in expression",
    `function f(x: int, y: boolean): int {}
    let g = f;
    print(g(1, true));
    f = g; // Type check here`,
  ],
  [
    "pass a function to a function",
    `function f(x: int, y: (boolean)->void): int { return 1; }
     function g(z: boolean) {}
     f(2, g);`,
  ],
  [
    "function return types",
    `function square(x: int): int { return x * x; }
     function compose(): (int)->int { return square; }`,
  ],
  ["function assign", "function f() {} let g = f; let h = [g, f]; print(h[0]());"],
  ["struct parameters", "struct S {} function f(x: S) {}"],
  ["array parameters", "function f(x: [int?]) {}"],
  ["optional parameters", "function f(x: [int], y: string?) {}"],
  ["empty optional types", "print(no [int]); print(no string);"],
  ["types in function type", "function f(g: (int?, float)->string) {}"],
  ["voids in fn type", "function f(g: (void)->void) {}"],
  ["outer variable", "let x=1; while(false) {print(x);}"],
  ["built-in constants", "print(25.0 * π);"],
  ["built-in sin", "print(sin(π));"],
  ["built-in cos", "print(cos(93.999));"],
  ["built-in hypot", "print(hypot(-4.0, 3.00001));"],
]

// Programs that are syntactically correct but have semantic errors
const semanticErrors = [
  ["non-distinct fields", "struct S {x: boolean x: int}", /Fields must be distinct/],
  ["non-int increment", "let x=false;x++;", /an integer/],
  ["non-int decrement", 'let x=some[""];x++;', /an integer/],
  ["undeclared id", "print(x);", /Identifier x not declared/],
  ["redeclared id", "let x = 1;let x = 1;", /Identifier x already declared/],
  ["recursive struct", "struct S { x: int y: S }", /must not be self-containing/],
```

```
["assign to const", "const x = 1;x = 2;", /Cannot assign to constant/],
["assign bad type", "let x=1;x=true;", /Cannot assign a boolean to a int/],
["assign bad array type", "let x=1;x=[true];", /Cannot assign a \[boolean\] to a int/],
["assign bad optional type", "let x=1;x=some 2;", /Cannot assign a int\? to a int/],
["break outside loop", "break;", /Break can only appear in a loop/],
[
  "break inside function",
  "while true {function f() {break;}}",
  /Break can only appear in a loop/,
],
["return outside function", "return;", /Return can only appear in a function/],
[
  "return value from void function",
  "function f() {return 1;}",
  /Cannot return a value/,
],
["return nothing from non-void", "function f(): int {return;}", /should be returned/],
["return type mismatch", "function f(): int {return false;}", /boolean to a int/],
["non-boolean short if test", "if 1 {}", /Expected a boolean/],
["non-boolean if test", "if 1 {} else {}", /Expected a boolean/],
["non-boolean while test", "while 1 {}", /Expected a boolean/],
["non-integer repeat", 'repeat "1" {}', /Expected an integer/],
["non-integer low range", "for i in true...2 {}", /Expected an integer/],
["non-integer high range", "for i in 1..<no int {}", /Expected an integer/],
["non-array in for", "for i in 100 {}", /Expected an array/],
["non-boolean conditional test", "print(1?2:3);", /Expected a boolean/],
["diff types in conditional arms", "print(true?1:true);", /not have the same type/],
["unwrap non-optional", "print(1??2);", /Expected an optional/],
["bad types for ||", "print(false||1);", /Expected a boolean/],
["bad types for &&", "print(false&&1);", /Expected a boolean/],
["bad types for ==", "print(false==1);", /Operands do not have the same type/],
["bad types for !=", "print(false==1);", /Operands do not have the same type/],
["bad types for +", "print(false+1);", /Expected a number or string/],
["bad types for -", "print(false-1);", /Expected a number/],
["bad types for *", "print(false*1);", /Expected a number/],
["bad types for /", "print(false/1);", /Expected a number/],
["bad types for **", "print(false**1);", /Expected a number/],
["bad types for <", "print(false<1);", /Expected a number or string/],
["bad types for <=", "print(false<=1);", /Expected a number or string/],
["bad types for >", "print(false>1);", /Expected a number or string/],
["bad types for >=", "print(false>=1);", /Expected a number or string/],
["bad types for ==", "print(2==2.0);", /not have the same type/],
["bad types for !=", "print(false!=1);", /not have the same type/],
["bad types for negation", "print(-true);", /Expected a number/],
["bad types for length", "print(#false);", /Expected an array/],
["bad types for not", 'print(!"hello");', /Expected a boolean/],
["bad types for random", "print(random 3);", /Expected an array/],
["non-integer index", "let a=[1];print(a[false]);", /Expected an integer/],
```

```
  ["no such field", "struct S{} let x=S(); print(x.y);", /No such field/],
  ["diff type array elements", "print([3,3.0]);", /Not all elements have the same type/],
  ["shadowing", "let x = 1;\nwhile true {let x = 1;}", /Identifier x already declared/],
  ["call of uncallable", "let x = 1;\nprint(x());", /Call of non-function/],
  [
    "Too many args",
    "function f(x: int) {}\nf(1,2);",
    /1 argument\(s\) required but 2 passed/,
  ],
  [
    "Too few args",
    "function f(x: int) {}\nf();",
    /1 argument\(s\) required but 0 passed/,
  ],
  [
    "Parameter type mismatch",
    "function f(x: int) {}\nf(false);",
    /Cannot assign a boolean to a int/,
  ],
  [
    "function type mismatch",
    `function f(x: int, y: (boolean)->void): int { return 1; }
     function g(z: boolean): int { return 5; }
     f(2, g);`,
    /Cannot assign a \(boolean\)->int to a \(boolean\)->void/,
  ],
  ["bad param type in fn assign", "function f(x: int) {} function g(y: float) {} f = g;"],
  [
    "bad return type in fn assign",
    'function f(x: int): int {return 1;} function g(y: int): string {return "uh-oh";} f = g;
    /Cannot assign a \(int\)->string to a \(int\)->int/,
  ],
  ["bad call to sin()", "print(sin(true));", /Cannot assign a boolean to a float/],
  ["Non-type in param", "let x=1;function f(y:x){}", /Type expected/],
  ["Non-type in return type", "let x=1;function f():x{return 1;}", /Type expected/],
  ["Non-type in field type", "let x=1;struct S {y:x}", /Type expected/],
]

describe("The analyzer", () => {
  for (const [scenario, source] of semanticChecks) {
    it(`recognizes ${scenario}`, () => {
      assert.ok(analyze(parse(source)))
    })
  }
  for (const [scenario, source, errorMessagePattern] of semanticErrors) {
    it(`throws on ${scenario}`, () => {
      assert.throws(() => analyze(parse(source)), errorMessagePattern)
    })
```

```
  }
  it("produces the expected representation for a trivial program", () => {
    assert.deepEqual(
      analyze(parse("let x = π + 2.2;")),
      program([
        variableDeclaration(
          variable("x", false, floatType),
          binary("+", variable("π", true, floatType), 2.2, floatType)
        ),
      ])
    )
  })
})
```

Study these tests, and if you have the inclination, write some of your own.

Of course, if you ran `npm test` now, you'd get errors since we haven't yet implemented the analyzer itself. That's next.

Writing the Semantic Analyzer

The analyzer is, well, really big. But we'll do our best to structure it in manageable pieces. Here's a big picture view of the file:

```
class Context { /* ... */ }

export default function analyze(match) {
  /* a local variable to hold the current context */

  /* a bunch of semantic validation functions */

  /* Definitions of the semantic actions */

  /* One line to run it */
}
```

Here's the Carlos analyzer. It's a big module. There is so much to explain. Rather than using the guide to explain what is going on, we'll defer you to the comments within the source code below:

analyzer.js

```
// The semantic analyzer exports a single function, analyze(match), that
// accepts a grammar match object (the CST) from Ohm and produces the
// internal representation of the program (pretty close to what is usually
// called the AST). This representation also includes entities from the
// standard library, as needed.
```

```
import * as core from "./core.js"

// A few declarations to save typing
const INT = core.intType
const FLOAT = core.floatType
const STRING = core.stringType
const BOOLEAN = core.boolType
const ANY = core.anyType
const VOID = core.voidType

class Context {
  // Like most statically-scoped languages, Carlos contexts will contain a
  // map for their locally declared identifiers and a reference to the parent
  // context. The parent of the global context is null. In addition, the
  // context records whether analysis is current within a loop (so we can
  // properly check break statements), and reference to the current function
  // (so we can properly check return statements).
  constructor({ parent = null, locals = new Map(), inLoop = false, function: f = null }) {
    Object.assign(this, { parent, locals, inLoop, function: f })
  }
  add(name, entity) {
    this.locals.set(name, entity)
  }
  lookup(name) {
    return this.locals.get(name) || this.parent?.lookup(name)
  }
  static root() {
    return new Context({ locals: new Map(Object.entries(core.standardLibrary)) })
  }
  newChildContext(props) {
    return new Context({ ...this, ...props, parent: this, locals: new Map() })
  }
}

export default function analyze(match) {
  // Track the context manually via a simple variable. The initial context
  // contains the mappings from the standard library. Add to this context
  // as necessary. When needing to descent into a new scope, create a new
  // context with the current context as its parent. When leaving a scope,
  // reset this variable to the parent context.
  let context = Context.root()

  // The single gate for error checking. Pass in a condition that must be true.
  // Use errorLocation to give contextual information about the error that will
  // appear: this should be an object whose "at" property is a parse tree node.
  // Ohm's getLineAndColumnMessage will be used to prefix the error message. This
  // allows any semantic analysis errors to be presented to an end user in the
  // same format as Ohm's reporting of syntax errors.
```

```
function must(condition, message, errorLocation) {
  if (!condition) {
    const prefix = errorLocation.at.source.getLineAndColumnMessage()
    throw new Error(`${prefix}${message}`)
  }
}

// Next come a number of carefully named utility functions that keep the
// analysis code clean and readable. Without these utilities, the analysis
// code would be cluttered with if-statements and error messages. Each of
// the utilities accept a parameter that should be an object with an "at"
// property that is a parse tree node. This is used to provide contextual
// information in the error message.

function mustNotAlreadyBeDeclared(name, at) {
  must(!context.lookup(name), `Identifier ${name} already declared`, at)
}

function mustHaveBeenFound(entity, name, at) {
  must(entity, `Identifier ${name} not declared`, at)
}

function mustHaveNumericType(e, at) {
  must([INT, FLOAT].includes(e.type), "Expected a number", at)
}

function mustHaveNumericOrStringType(e, at) {
  must([INT, FLOAT, STRING].includes(e.type), "Expected a number or string", at)
}

function mustHaveBooleanType(e, at) {
  must(e.type === BOOLEAN, "Expected a boolean", at)
}

function mustHaveIntegerType(e, at) {
  must(e.type === INT, "Expected an integer", at)
}

function mustHaveAnArrayType(e, at) {
  must(e.type?.kind === "ArrayType", "Expected an array", at)
}

function mustHaveAnOptionalType(e, at) {
  must(e.type?.kind === "OptionalType", "Expected an optional", at)
}

function mustHaveAStructType(e, at) {
  must(e.type?.kind === "StructType", "Expected a struct", at)
```

```
}

function mustHaveAnOptionalStructType(e, at) {
  // Used to check e?.x expressions, e must be an optional struct
  must(
    e.type?.kind === "OptionalType" && e.type.baseType?.kind === "StructType",
    "Expected an optional struct",
    at
  )
}

function mustBothHaveTheSameType(e1, e2, at) {
  must(equivalent(e1.type, e2.type), "Operands do not have the same type", at)
}

function mustAllHaveSameType(expressions, at) {
  // Used to check the elements of an array expression, and the two
  // arms of a conditional expression, among other scenarios.
  must(
    expressions.slice(1).every(e => equivalent(e.type, expressions[0].type)),
    "Not all elements have the same type",
    at
  )
}

function mustBeAType(e, at) {
  // This is a rather ugly hack
  must(e?.kind.endsWith("Type"), "Type expected", at)
}

function mustBeAnArrayType(t, at) {
  must(t?.kind === "ArrayType", "Must be an array type", at)
}

function includesAsField(structType, type) {
  // Whether the struct type has a field of type type, directly or indirectly
  return structType.fields.some(
    field =>
      field.type === type ||
      (field.type?.kind === "StructType" && includesAsField(field.type, type))
  )
}

function mustNotBeSelfContaining(structType, at) {
  const containsSelf = includesAsField(structType, structType)
  must(!containsSelf, "Struct type must not be self-containing", at)
}
```

```
function equivalent(t1, t2) {
  return (
    t1 === t2 ||
    (t1?.kind === "OptionalType" &&
      t2?.kind === "OptionalType" &&
      equivalent(t1.baseType, t2.baseType)) ||
    (t1?.kind === "ArrayType" &&
      t2?.kind === "ArrayType" &&
      equivalent(t1.baseType, t2.baseType)) ||
    (t1?.kind === "FunctionType" &&
      t2?.kind === "FunctionType" &&
      equivalent(t1.returnType, t2.returnType) &&
      t1.paramTypes.length === t2.paramTypes.length &&
      t1.paramTypes.every((t, i) => equivalent(t, t2.paramTypes[i])))
  )
}

function assignable(fromType, toType) {
  return (
    toType == ANY ||
    equivalent(fromType, toType) ||
    (fromType?.kind === "FunctionType" &&
      toType?.kind === "FunctionType" &&
      // covariant in return types
      assignable(fromType.returnType, toType.returnType) &&
      fromType.paramTypes.length === toType.paramTypes.length &&
      // contravariant in parameter types
      toType.paramTypes.every((t, i) => assignable(t, fromType.paramTypes[i])))
  )
}

function typeDescription(type) {
  switch (type.kind) {
    case "IntType":
      return "int"
    case "FloatType":
      return "float"
    case "StringType":
      return "string"
    case "BoolType":
      return "boolean"
    case "VoidType":
      return "void"
    case "AnyType":
      return "any"
    case "StructType":
      return type.name
    case "FunctionType":
```

```
      const paramTypes = type.paramTypes.map(typeDescription).join(", ")
      const returnType = typeDescription(type.returnType)
      return `(${paramTypes})->${returnType}`
    case "ArrayType":
      return `[${typeDescription(type.baseType)}]`
    case "OptionalType":
      return `${typeDescription(type.baseType)}?`
  }
}

function mustBeAssignable(e, { toType: type }, at) {
  const message = `Cannot assign a ${typeDescription(e.type)} to a ${typeDescription(
    type
  )}`
  must(assignable(e.type, type), message, at)
}

function mustNotBeReadOnly(e, at) {
  must(!e.readOnly, `Cannot assign to constant ${e.name}`, at)
}

function mustHaveDistinctFields(type, at) {
  const fieldNames = new Set(type.fields.map(f => f.name))
  must(fieldNames.size === type.fields.length, "Fields must be distinct", at)
}

function mustHaveMember(structType, field, at) {
  must(structType.fields.map(f => f.name).includes(field), "No such field", at)
}

function mustBeInLoop(at) {
  must(context.inLoop, "Break can only appear in a loop", at)
}

function mustBeInAFunction(at) {
  must(context.function, "Return can only appear in a function", at)
}

function mustBeCallable(e, at) {
  const callable = e?.kind === "StructType" || e.type?.kind === "FunctionType"
  must(callable, "Call of non-function or non-constructor", at)
}

function mustNotReturnAnything(f, at) {
  must(f.type.returnType === VOID, "Something should be returned", at)
}

function mustReturnSomething(f, at) {
```

```
  must(f.type.returnType !== VOID, "Cannot return a value from this function", at)
}

function mustBeReturnable(e, { from: f }, at) {
  mustBeAssignable(e, { toType: f.type.returnType }, at)
}

function mustHaveCorrectArgumentCount(argCount, paramCount, at) {
  const message = `${paramCount} argument(s) required but ${argCount} passed`
  must(argCount === paramCount, message, at)
}

// Building the program representation will be done together with semantic
// analysis and error checking. In Ohm, we do this with a semantics object
// that has an operation for each relevant rule in the grammar. Since the
// purpose of analysis is to build the program representation, we will name
// the operations "rep" for "representation". Most of the rules are straight-
// forward except for those dealing with function and type declarations,
// since types and functions need to be dealt with in two steps to allow
// recursion.
const builder = match.matcher.grammar.createSemantics().addOperation("rep", {
  Program(statements) {
    return core.program(statements.children.map(s => s.rep()))
  },

  VarDecl(modifier, id, _eq, exp, _semicolon) {
    const initializer = exp.rep()
    const readOnly = modifier.sourceString === "const"
    const variable = core.variable(id.sourceString, readOnly, initializer.type)
    mustNotAlreadyBeDeclared(id.sourceString, { at: id })
    context.add(id.sourceString, variable)
    return core.variableDeclaration(variable, initializer)
  },

  TypeDecl(_struct, id, _left, fields, _right) {
    // To allow recursion, enter into context without any fields yet
    const type = core.structType(id.sourceString, [])
    mustNotAlreadyBeDeclared(id.sourceString, { at: id })
    context.add(id.sourceString, type)
    // Now add the types as you parse and analyze. Since we already added
    // the struct type itself into the context, we can use it in fields.
    type.fields = fields.children.map(field => field.rep())
    mustHaveDistinctFields(type, { at: id })
    mustNotBeSelfContaining(type, { at: id })
    return core.typeDeclaration(type)
  },

  Field(id, _colon, type) {
```

```
  return core.field(id.sourceString, type.rep())
},

FunDecl(_fun, id, parameters, _colons, type, block) {
  // Start by making the function, but we don't yet know its type.
  // Also add it to the context so that we can have recursion.
  const fun = core.fun(id.sourceString)
  mustNotAlreadyBeDeclared(id.sourceString, { at: id })
  context.add(id.sourceString, fun)

  // Parameters are part of the child context
  context = context.newChildContext({ inLoop: false, function: fun })
  const params = parameters.rep()

  // Now that the parameters are known, we compute the function's type.
  // This is fine; we did not need the type to analyze the parameters,
  // but we do need to set it before analyzing the body.
  const paramTypes = params.map(param => param.type)
  const returnType = type.children?.[0]?.rep() ?? VOID
  fun.type = core.functionType(paramTypes, returnType)

  // Analyze body while still in child context
  const body = block.rep()

  // Go back up to the outer context before returning
  context = context.parent
  return core.functionDeclaration(fun, params, body)
},

Params(_open, paramList, _close) {
  // Returns a list of variable nodes
  return paramList.asIteration().children.map(p => p.rep())
},

Param(id, _colon, type) {
  const param = core.variable(id.sourceString, false, type.rep())
  mustNotAlreadyBeDeclared(param.name, { at: id })
  context.add(param.name, param)
  return param
},

Type_optional(baseType, _questionMark) {
  return core.optionalType(baseType.rep())
},

Type_array(_left, baseType, _right) {
  return core.arrayType(baseType.rep())
},
```

```
Type_function(_left, types, _right, _arrow, type) {
  const paramTypes = types.asIteration().children.map(t => t.rep())
  const returnType = type.rep()
  return core.functionType(paramTypes, returnType)
},

Type_id(id) {
  const entity = context.lookup(id.sourceString)
  mustHaveBeenFound(entity, id.sourceString, { at: id })
  mustBeAType(entity, { at: id })
  return entity
},

Statement_bump(exp, operator, _semicolon) {
  const variable = exp.rep()
  mustHaveIntegerType(variable, { at: exp })
  return operator.sourceString === "++"
    ? core.increment(variable)
    : core.decrement(variable)
},

Statement_assign(variable, _eq, expression, _semicolon) {
  const source = expression.rep()
  const target = variable.rep()
  mustBeAssignable(source, { toType: target.type }, { at: variable })
  mustNotBeReadOnly(target, { at: variable })
  return core.assignment(target, source)
},

Statement_call(call, _semicolon) {
  return call.rep()
},

Statement_break(breakKeyword, _semicolon) {
  mustBeInLoop({ at: breakKeyword })
  return core.breakStatement
},

Statement_return(returnKeyword, exp, _semicolon) {
  mustBeInAFunction({ at: returnKeyword })
  mustReturnSomething(context.function, { at: returnKeyword })
  const returnExpression = exp.rep()
  mustBeReturnable(returnExpression, { from: context.function }, { at: exp })
  return core.returnStatement(returnExpression)
},

Statement_shortreturn(returnKeyword, _semicolon) {
```

```
  mustBeInAFunction({ at: returnKeyword })
  mustNotReturnAnything(context.function, { at: returnKeyword })
  return core.shortReturnStatement()
},

IfStmt_long(_if, exp, block1, _else, block2) {
  const test = exp.rep()
  mustHaveBooleanType(test, { at: exp })
  context = context.newChildContext()
  const consequent = block1.rep()
  context = context.parent
  context = context.newChildContext()
  const alternate = block2.rep()
  context = context.parent
  return core.ifStatement(test, consequent, alternate)
},

IfStmt_elsif(_if, exp, block, _else, trailingIfStatement) {
  const test = exp.rep()
  mustHaveBooleanType(test, { at: exp })
  context = context.newChildContext()
  const consequent = block.rep()
  context = context.parent
  const alternate = trailingIfStatement.rep()
  return core.ifStatement(test, consequent, alternate)
},

IfStmt_short(_if, exp, block) {
  const test = exp.rep()
  mustHaveBooleanType(test, { at: exp })
  context = context.newChildContext()
  const consequent = block.rep()
  context = context.parent
  return core.shortIfStatement(test, consequent)
},

LoopStmt_while(_while, exp, block) {
  const test = exp.rep()
  mustHaveBooleanType(test, { at: exp })
  context = context.newChildContext({ inLoop: true })
  const body = block.rep()
  context = context.parent
  return core.whileStatement(test, body)
},

LoopStmt_repeat(_repeat, exp, block) {
  const count = exp.rep()
  mustHaveIntegerType(count, { at: exp })
```

```
  context = context.newChildContext({ inLoop: true })
  const body = block.rep()
  context = context.parent
  return core.repeatStatement(count, body)
},

LoopStmt_range(_for, id, _in, exp1, op, exp2, block) {
  const [low, high] = [exp1.rep(), exp2.rep()]
  mustHaveIntegerType(low, { at: exp1 })
  mustHaveIntegerType(high, { at: exp2 })
  const iterator = core.variable(id.sourceString, true, INT)
  context = context.newChildContext({ inLoop: true })
  context.add(id.sourceString, iterator)
  const body = block.rep()
  context = context.parent
  return core.forRangeStatement(iterator, low, op.sourceString, high, body)
},

LoopStmt_collection(_for, id, _in, exp, block) {
  const collection = exp.rep()
  mustHaveAnArrayType(collection, { at: exp })
  const iterator = core.variable(id.sourceString, true, collection.type.baseType)
  context = context.newChildContext({ inLoop: true })
  context.add(iterator.name, iterator)
  const body = block.rep()
  context = context.parent
  return core.forStatement(iterator, collection, body)
},

Block(_open, statements, _close) {
  // No need for a block node, just return the list of statements
  return statements.children.map(s => s.rep())
},

Exp_conditional(exp, _questionMark, exp1, colon, exp2) {
  const test = exp.rep()
  mustHaveBooleanType(test, { at: exp })
  const [consequent, alternate] = [exp1.rep(), exp2.rep()]
  mustBothHaveTheSameType(consequent, alternate, { at: colon })
  return core.conditional(test, consequent, alternate, consequent.type)
},

Exp1_unwrapelse(exp1, elseOp, exp2) {
  const [optional, op, alternate] = [exp1.rep(), elseOp.sourceString, exp2.rep()]
  mustHaveAnOptionalType(optional, { at: exp1 })
  mustBeAssignable(alternate, { toType: optional.type.baseType }, { at: exp2 })
  return core.binary(op, optional, alternate, optional.type)
},
```

```
Exp2_or(exp, _ops, exps) {
  let left = exp.rep()
  mustHaveBooleanType(left, { at: exp })
  for (let e of exps.children) {
    let right = e.rep()
    mustHaveBooleanType(right, { at: e })
    left = core.binary("||", left, right, BOOLEAN)
  }
  return left
},

Exp2_and(exp, _ops, exps) {
  let left = exp.rep()
  mustHaveBooleanType(left, { at: exp })
  for (let e of exps.children) {
    let right = e.rep()
    mustHaveBooleanType(right, { at: e })
    left = core.binary("&&", left, right, BOOLEAN)
  }
  return left
},

Exp3_bitor(exp, _ops, exps) {
  let left = exp.rep()
  mustHaveIntegerType(left, { at: exp })
  for (let e of exps.children) {
    let right = e.rep()
    mustHaveIntegerType(right, { at: e })
    left = core.binary("|", left, right, INT)
  }
  return left
},

Exp3_bitxor(exp, xorOps, exps) {
  let left = exp.rep()
  mustHaveIntegerType(left, { at: exp })
  for (let e of exps.children) {
    let right = e.rep()
    mustHaveIntegerType(right, { at: e })
    left = core.binary("^", left, right, INT)
  }
  return left
},

Exp3_bitand(exp, andOps, exps) {
  let left = exp.rep()
  mustHaveIntegerType(left, { at: exp })
```

```
  for (let e of exps.children) {
    let right = e.rep()
    mustHaveIntegerType(right, { at: e })
    left = core.binary("&", left, right, INT)
  }
  return left
},

Exp4_compare(exp1, relop, exp2) {
  const [left, op, right] = [exp1.rep(), relop.sourceString, exp2.rep()]
  // == and != can have any operand types as long as they are the same
  // But inequality operators can only be applied to numbers and strings
  if (["<", "<=", ">", ">="].includes(op)) {
    mustHaveNumericOrStringType(left, { at: exp1 })
  }
  mustBothHaveTheSameType(left, right, { at: relop })
  return core.binary(op, left, right, BOOLEAN)
},

Exp5_shift(exp1, shiftOp, exp2) {
  const [left, op, right] = [exp1.rep(), shiftOp.sourceString, exp2.rep()]
  mustHaveIntegerType(left, { at: exp1 })
  mustHaveIntegerType(right, { at: exp2 })
  return core.binary(op, left, right, INT)
},

Exp6_add(exp1, addOp, exp2) {
  const [left, op, right] = [exp1.rep(), addOp.sourceString, exp2.rep()]
  if (op === "+") {
    mustHaveNumericOrStringType(left, { at: exp1 })
  } else {
    mustHaveNumericType(left, { at: exp1 })
  }
  mustBothHaveTheSameType(left, right, { at: addOp })
  return core.binary(op, left, right, left.type)
},

Exp7_multiply(exp1, mulOp, exp2) {
  const [left, op, right] = [exp1.rep(), mulOp.sourceString, exp2.rep()]
  mustHaveNumericType(left, { at: exp1 })
  mustBothHaveTheSameType(left, right, { at: mulOp })
  return core.binary(op, left, right, left.type)
},

Exp8_power(exp1, powerOp, exp2) {
  const [left, op, right] = [exp1.rep(), powerOp.sourceString, exp2.rep()]
  mustHaveNumericType(left, { at: exp1 })
  mustBothHaveTheSameType(left, right, { at: powerOp })
```

```
  return core.binary(op, left, right, left.type)
},

Exp8_unary(unaryOp, exp) {
  const [op, operand] = [unaryOp.sourceString, exp.rep()]
  let type
  if (op === "#") {
    mustHaveAnArrayType(operand, { at: exp })
    type = INT
  } else if (op === "-") {
    mustHaveNumericType(operand, { at: exp })
    type = operand.type
  } else if (op === "!") {
    mustHaveBooleanType(operand, { at: exp })
    type = BOOLEAN
  } else if (op === "some") {
    type = core.optionalType(operand.type)
  } else if (op === "random") {
    mustHaveAnArrayType(operand, { at: exp })
    type = operand.type.baseType
  }
  return core.unary(op, operand, type)
},

Exp9_emptyarray(ty, _open, _close) {
  const type = ty.rep()
  mustBeAnArrayType(type, { at: ty })
  return core.emptyArray(type)
},

Exp9_arrayexp(_open, args, _close) {
  const elements = args.asIteration().children.map(e => e.rep())
  mustAllHaveSameType(elements, { at: args })
  return core.arrayExpression(elements)
},

Exp9_emptyopt(_no, type) {
  return core.emptyOptional(type.rep())
},

Exp9_parens(_open, expression, _close) {
  return expression.rep()
},

Exp9_subscript(exp1, _open, exp2, _close) {
  const [array, subscript] = [exp1.rep(), exp2.rep()]
  mustHaveAnArrayType(array, { at: exp1 })
  mustHaveIntegerType(subscript, { at: exp2 })
```

```
  return core.subscript(array, subscript)
},

Exp9_member(exp, dot, id) {
  const object = exp.rep()
  let structType
  if (dot.sourceString === "?.") {
    mustHaveAnOptionalStructType(object, { at: exp })
    structType = object.type.baseType
  } else {
    mustHaveAStructType(object, { at: exp })
    structType = object.type
  }
  mustHaveMember(structType, id.sourceString, { at: id })
  const field = structType.fields.find(f => f.name === id.sourceString)
  return core.memberExpression(object, dot.sourceString, field)
},

Exp9_call(exp, open, expList, _close) {
  const callee = exp.rep()
  mustBeCallable(callee, { at: exp })
  const exps = expList.asIteration().children
  const targetTypes =
    callee?.kind === "StructType"
      ? callee.fields.map(f => f.type)
      : callee.type.paramTypes
  mustHaveCorrectArgumentCount(exps.length, targetTypes.length, { at: open })
  const args = exps.map((exp, i) => {
    const arg = exp.rep()
    mustBeAssignable(arg, { toType: targetTypes[i] }, { at: exp })
    return arg
  })
  return callee?.kind === "StructType"
    ? core.constructorCall(callee, args)
    : core.functionCall(callee, args)
},

Exp9_id(id) {
  // When an id appears in an expression, it had better have been declared
  const entity = context.lookup(id.sourceString)
  mustHaveBeenFound(entity, id.sourceString, { at: id })
  return entity
},

true(_) {
  return true
},
```

```
    false(_) {
      return false
    },

    intlit(_digits) {
      // Carlos ints will be represented as plain JS bigints
      return BigInt(this.sourceString)
    },

    floatlit(_whole, _point, _fraction, _e, _sign, _exponent) {
      // Carlos floats will be represented as plain JS numbers
      return Number(this.sourceString)
    },

    stringlit(_openQuote, _chars, _closeQuote) {
      // Carlos strings will be represented as plain JS strings, including
      // the quotation marks
      return this.sourceString
    },
  })

  return builder(match).rep()
}
```

***Exercise**: In more complex languages you were need additional context fields, such as the "current class" or "current namespace." What other items might be required? Give examples from real languages that you know.*

> ***What was this module so large?***
>
> *In some sense, the size of this module compared to the others isn't really too surprising. Syntax was much easier: grammar processing is a well-understood process, so we get great libraries like Ohm that can parse pretty much any language. But semantic processing is language-dependent, context-sensitive, and full of sometimes messy and crazy rules.*

At this point:

- ✅ Fill in *src/analyzer.js*.
- ✅ Run `npm test` and make sure we've satisfied the tests we wrote earlier.
- ✅ Run `node src/carlos.js examples/gcd.carlos analyzed` and make sure the output looks good. You should now be able to see graph-stringify in action.
- ✅ Try a few more examples an explore the representations that you can now build.
- ✅ git add, commit, and push.

Optimization

In practice, you do optimizations, or code improvement, at several different places throughout a compiler. But many compilers have a special optimization pass that runs on the program representation, right after semantic analysis. A simple optimizer includes strength reductions, constant folding, dead code elimination for while-false and simplifications for if-true and if-false. Tests first:

optimizer.test.js

```
import { describe, it } from "node:test"
import assert from "node:assert/strict"
import optimize from "../src/optimizer.js"
import * as core from "../src/core.js"

// Make some test cases easier to read
const x = core.variable("x", false, core.intType)
const a = core.variable("a", false, core.arrayType(core.intType))
const xpp = core.increment(x)
const xmm = core.decrement(x)
const return1p1 = core.returnStatement(core.binary("+", 1, 1, core.intType))
const return2 = core.returnStatement(2)
const returnX = core.returnStatement(x)
const onePlusTwo = core.binary("+", 1, 2, core.intType)
const identity = Object.assign(core.fun("id", core.anyType), { body: returnX })
const voidInt = core.functionType([], core.intType)
const intFun = body => core.functionDeclaration("f", core.fun("f", voidInt), [], [body])
const callIdentity = args => core.functionCall(identity, args)
const or = (...d) => d.reduce((x, y) => core.binary("||", x, y))
const and = (...c) => c.reduce((x, y) => core.binary("&&", x, y))
const less = (x, y) => core.binary("<", x, y)
const eq = (x, y) => core.binary("==", x, y)
const times = (x, y) => core.binary("*", x, y)
const neg = x => core.unary("-", x)
const array = (...elements) => core.arrayExpression(elements)
const assign = (v, e) => core.assignment(v, e)
const emptyArray = core.emptyArray(core.intType)
const sub = (a, e) => core.subscript(a, e)
const unwrapElse = (o, e) => core.binary("??", o, e)
const emptyOptional = core.emptyOptional(core.intType)
const some = x => core.unary("some", x)
const program = core.program

const tests = [
  ["folds +", core.binary("+", 5, 8), 13],
  ["folds -", core.binary("-", 5n, 8n), -3n],
  ["folds *", core.binary("*", 5, 8), 40],
  ["folds /", core.binary("/", 5, 8), 0.625],
  ["folds **", core.binary("**", 5, 8), 390625],
  ["folds <", core.binary("<", 5, 8), true],
  ["folds <=", core.binary("<=", 5, 8), true],
  ["folds ==", core.binary("==", 5, 8), false],
```

```
["folds !=", core.binary("!=", 5, 8), true],
["folds >=", core.binary(">=", 5, 8), false],
["folds >", core.binary(">", 5, 8), false],
["optimizes +0", core.binary("+", x, 0), x],
["optimizes -0", core.binary("-", x, 0), x],
["optimizes *1", core.binary("*", x, 1), x],
["optimizes /1", core.binary("/", x, 1), x],
["optimizes *0", core.binary("*", x, 0), 0],
["optimizes 0*", core.binary("*", 0, x), 0],
["optimizes 0/", core.binary("/", 0, x), 0],
["optimizes 0+", core.binary("+", 0, x), x],
["optimizes 0-", core.binary("-", 0, x), neg(x)],
["optimizes 1*", core.binary("*", 1, x), x],
["folds negation", core.unary("-", 8), -8],
["optimizes 1**", core.binary("**", 1, x), 1],
["optimizes **0", core.binary("**", x, 0), 1],
["removes left false from ||", or(false, less(x, 1)), less(x, 1)],
["removes right false from ||", or(less(x, 1), false), less(x, 1)],
["removes left true from &&", and(true, less(x, 1)), less(x, 1)],
["removes right true from &&", and(less(x, 1), true), less(x, 1)],
["removes x=x at beginning", program([core.assignment(x, x), xpp]), program([xpp])],
["removes x=x at end", program([xpp, core.assignment(x, x)]), program([xpp])],
["removes x=x in middle", program([xpp, assign(x, x), xpp]), program([xpp, xpp])],
["optimizes if-true", core.ifStatement(true, [xpp], []), [xpp]],
["optimizes if-false", core.ifStatement(false, [], [xpp]), [xpp]],
["optimizes short-if-true", core.shortIfStatement(true, [xmm]), [xmm]],
["optimizes short-if-false", core.shortIfStatement(false, [xpp]), []],
["optimizes while-false", program([core.whileStatement(false, [xpp])]), program([])],
["optimizes repeat-0", program([core.repeatStatement(0, [xpp])]), program([])],
["optimizes for-range", core.forRangeStatement(x, 5, "...", 3, [xpp]), []],
["optimizes for-empty-array", core.forStatement(x, emptyArray, [xpp]), []],
["applies if-false after folding", core.shortIfStatement(eq(1, 1), [xpp]), [xpp]],
["optimizes away nil", unwrapElse(emptyOptional, 3), 3],
["optimizes left conditional true", core.conditional(true, 55, 89), 55],
["optimizes left conditional false", core.conditional(false, 55, 89), 89],
["optimizes in functions", program([intFun(return1p1)]), program([intFun(return2)])],
["optimizes in subscripts", sub(a, onePlusTwo), sub(a, 3)],
["optimizes in array literals", array(0, onePlusTwo, 9), array(0, 3, 9)],
["optimizes in arguments", callIdentity([times(3, 5)]), callIdentity([15])],
[
  "passes through nonoptimizable constructs",
  ...Array(2).fill([
    core.program([core.shortReturnStatement()]),
    core.variableDeclaration("x", true, "z"),
    core.typeDeclaration([core.field("x", core.intType)]),
    core.assignment(x, core.binary("*", x, "z")),
    core.assignment(x, core.unary("not", x)),
    core.constructorCall(identity, core.memberExpression(x, ".", "f")),
```

```
      core.variableDeclaration("q", false, core.emptyArray(core.floatType)),
      core.variableDeclaration("r", false, core.emptyOptional(core.intType)),
      core.whileStatement(true, [core.breakStatement]),
      core.repeatStatement(5, [core.returnStatement(1)]),
      core.conditional(x, 1, 2),
      unwrapElse(some(x), 7),
      core.ifStatement(x, [], []),
      core.shortIfStatement(x, []),
      core.forRangeStatement(x, 2, "..<", 5, []),
      core.forStatement(x, array(1, 2, 3), []),
    ]),
  ],
]

describe("The optimizer", () => {
  for (const [scenario, before, after] of tests) {
    it(`${scenario}`, () => {
      assert.deepEqual(optimize(before), after)
    })
  }
})
```

and the actual optimizer:

optimizer.js

```
// The optimizer module exports a single function, optimize(node), to perform
// machine-independent optimizations on the analyzed semantic representation.
//
// The only optimizations supported here are:
//
//   - assignments to self (x = x) turn into no-ops
//   - constant folding
//   - some strength reductions (+0, -0, *0, *1, etc.)
//   - turn references to built-ins true and false to be literals
//   - remove all disjuncts in || list after literal true
//   - remove all conjuncts in && list after literal false
//   - while-false becomes a no-op
//   - repeat-0 is a no-op
//   - for-loop over empty array is a no-op
//   - for-loop with low > high is a no-op
//   - if-true and if-false reduce to only the taken arm
//
// The optimizer also replaces token references with their actual values,
// since the original token line and column numbers are no longer needed.
// This simplifies code generation.

import * as core from "./core.js"
```

```
export default function optimize(node) {
  return optimizers?.[node.kind]?.(node) ?? node
}

const optimizers = {
  Program(p) {
    p.statements = p.statements.flatMap(optimize)
    return p
  },
  VariableDeclaration(d) {
    d.variable = optimize(d.variable)
    d.initializer = optimize(d.initializer)
    return d
  },
  TypeDeclaration(d) {
    d.type = optimize(d.type)
    return d
  },
  FunctionDeclaration(d) {
    d.fun = optimize(d.fun)
    if (d.body) d.body = d.body.flatMap(optimize)
    return d
  },
  Increment(s) {
    s.variable = optimize(s.variable)
    return s
  },
  Decrement(s) {
    s.variable = optimize(s.variable)
    return s
  },
  Assignment(s) {
    s.source = optimize(s.source)
    s.target = optimize(s.target)
    if (s.source === s.target) {
      return []
    }
    return s
  },
  BreakStatement(s) {
    return s
  },
  ReturnStatement(s) {
    s.expression = optimize(s.expression)
    return s
  },
  ShortReturnStatement(s) {
    return s
```

```
  },
  IfStatement(s) {
    s.test = optimize(s.test)
    s.consequent = s.consequent.flatMap(optimize)
    if (s.alternate?.kind?.endsWith?.("IfStatement")) {
      s.alternate = optimize(s.alternate)
    } else {
      s.alternate = s.alternate.flatMap(optimize)
    }
    if (s.test.constructor === Boolean) {
      return s.test ? s.consequent : s.alternate
    }
    return s
  },
  ShortIfStatement(s) {
    s.test = optimize(s.test)
    s.consequent = s.consequent.flatMap(optimize)
    if (s.test.constructor === Boolean) {
      return s.test ? s.consequent : []
    }
    return s
  },
  WhileStatement(s) {
    s.test = optimize(s.test)
    if (s.test === false) {
      // while false is a no-op
      return []
    }
    s.body = s.body.flatMap(optimize)
    return s
  },
  RepeatStatement(s) {
    s.count = optimize(s.count)
    if (s.count === 0) {
      // repeat 0 times is a no-op
      return []
    }
    s.body = s.body.flatMap(optimize)
    return s
  },
  ForRangeStatement(s) {
    s.iterator = optimize(s.iterator)
    s.low = optimize(s.low)
    s.op = optimize(s.op)
    s.high = optimize(s.high)
    s.body = s.body.flatMap(optimize)
    if (s.low.constructor === Number) {
      if (s.high.constructor === Number) {
```

```
      if (s.low > s.high) {
        return []
      }
    }
  }
  return s
},
ForStatement(s) {
  s.iterator = optimize(s.iterator)
  s.collection = optimize(s.collection)
  s.body = s.body.flatMap(optimize)
  if (s.collection?.kind === "EmptyArray") {
    return []
  }
  return s
},
Conditional(e) {
  e.test = optimize(e.test)
  e.consequent = optimize(e.consequent)
  e.alternate = optimize(e.alternate)
  if (e.test.constructor === Boolean) {
    return e.test ? e.consequent : e.alternate
  }
  return e
},
BinaryExpression(e) {
  e.op = optimize(e.op)
  e.left = optimize(e.left)
  e.right = optimize(e.right)
  if (e.op === "??") {
    // Coalesce empty optional unwraps
    if (e.left?.kind === "EmptyOptional") {
      return e.right
    }
  } else if (e.op === "&&") {
    // Optimize boolean constants in && and ||
    if (e.left === true) return e.right
    if (e.right === true) return e.left
  } else if (e.op === "||") {
    if (e.left === false) return e.right
    if (e.right === false) return e.left
  } else if ([Number, BigInt].includes(e.left.constructor)) {
    // Numeric constant folding when left operand is constant
    if ([Number, BigInt].includes(e.right.constructor)) {
      if (e.op === "+") return e.left + e.right
      if (e.op === "-") return e.left - e.right
      if (e.op === "*") return e.left * e.right
      if (e.op === "/") return e.left / e.right
```

```
      if (e.op === "**") return e.left ** e.right
      if (e.op === "<") return e.left < e.right
      if (e.op === "<=") return e.left <= e.right
      if (e.op === "==") return e.left === e.right
      if (e.op === "!=") return e.left !== e.right
      if (e.op === ">=") return e.left >= e.right
      if (e.op === ">") return e.left > e.right
    }
    if (e.left === 0 && e.op === "+") return e.right
    if (e.left === 1 && e.op === "*") return e.right
    if (e.left === 0 && e.op === "-") return core.unary("-", e.right)
    if (e.left === 1 && e.op === "**") return 1
    if (e.left === 0 && ["*", "/"].includes(e.op)) return 0
  } else if ([Number, BigInt].includes(e.right.constructor)) {
    // Numeric constant folding when right operand is constant
    if (["+", "-"].includes(e.op) && e.right === 0) return e.left
    if (["*", "/"].includes(e.op) && e.right === 1) return e.left
    if (e.op === "*" && e.right === 0) return 0
    if (e.op === "**" && e.right === 0) return 1
  }
  return e
},
UnaryExpression(e) {
  e.op = optimize(e.op)
  e.operand = optimize(e.operand)
  if (e.operand.constructor === Number) {
    if (e.op === "-") {
      return -e.operand
    }
  }
  return e
},
SubscriptExpression(e) {
  e.array = optimize(e.array)
  e.index = optimize(e.index)
  return e
},
ArrayExpression(e) {
  e.elements = e.elements.map(optimize)
  return e
},
MemberExpression(e) {
  e.object = optimize(e.object)
  return e
},
FunctionCall(c) {
  c.callee = optimize(c.callee)
  c.args = c.args.map(optimize)
```

```
    return c
  },
  ConstructorCall(c) {
    c.callee = optimize(c.callee)
    c.args = c.args.map(optimize)
    return c
  },
}
```

There's room for plenty more, but this is a good start.

At this point:

- ✅ Fill in *test/optimizer.test.js*
- ✅ Fill in *src/optimizer.js*.
- ✅ Run `npm test` and make sure we've satisfied the tests we wrote earlier.
- ✅ git add, commit, and push.

Generate Target Code

If all the semantic checks pass, we're ready to walk our semantic graph and output some code. We can output JavaScript, C, LLVM, some made-up intermediate language, some virtual machine language, or anything really. In these notes, we're going to output JavaScript.

There's no real magic here; things are pretty straightforward. But there are some useful things to keep in mind:

- Carlos and JavaScript have different reserved words. So we put a numeric suffix on each of the identifiers to distinguish them: for example, the Carlos variable `switch` must become something like `switch_1` in JavaScript. The mapping from Carlos entity to JavaScript identifier name is kept in a little cache in the code generator, so each Carlos variable gets a distinct JS name.
- Types are used only in the front-end of the compiler. One all the types are inferred, resolved, and checked, we know are Carlos program is type-safe. So we ignore them during code generation. Ditto for whether a variable is read-only or not.
- We'll do "inlining" on the built-in functions.

When generating JavaScript code, we don't have to expend any real effort into making the output pretty. If you wish to, you can bring in a third-party tool like js-beautify or see if you can get Prettier to work on your output.

As usual, start with the tests. We just pair example Carlos programs with the JS output we expect:

generator.test.js

```
import { describe, it } from "node:test"
import assert from "node:assert/strict"
import parse from "../src/parser.js"
import analyze from "../src/analyzer.js"
import optimize from "../src/optimizer.js"
```

```
import generate from "../src/generator.js"

function dedent(s) {
  return `${s}`.replace(/(?<=\n)\s+/g, "").trim()
}

const fixtures = [
  {
    name: "small",
    source: `
      let x = 3 * 7;
      x++;
      x--;
      let y = true;
      y = 5 ** -x / -100 > - x || false;
      print((y && y) || false || (x*2) != 5);
    `,
    expected: dedent`
      let x_1 = 21;
      x_1++;
      x_1--;
      let y_2 = true;
      y_2 = (((5 ** -(x_1)) / -(100)) > -(x_1));
      console.log(((y_2 && y_2) || ((x_1 * 2) !== 5)));
    `,
  },
  {
    name: "if",
    source: `
      let x = 0;
      if (x == 0) { print("1"); }
      if (x == 0) { print(1); } else { print(2); }
      if (x == 0) { print(1); } else if (x == 2) { print(3); }
      if (x == 0) { print(1); } else if (x == 2) { print(3); } else { print(4); }
    `,
    expected: dedent`
      let x_1 = 0;
      if ((x_1 === 0)) {
        console.log("1");
      }
      if ((x_1 === 0)) {
        console.log(1);
      } else {
        console.log(2);
      }
      if ((x_1 === 0)) {
        console.log(1);
      } else
```

```
      if ((x_1 === 2)) {
        console.log(3);
      }
    if ((x_1 === 0)) {
      console.log(1);
    } else
      if ((x_1 === 2)) {
        console.log(3);
      } else {
        console.log(4);
      }
  `,
},
{
  name: "while",
  source: `
    let x = 0;
    while x < 5 {
      let y = 0;
      while y < 5 {
        print(x * y);
        y = y + 1;
        break;
      }
      x = x + 1;
    }
  `,
  expected: dedent`
    let x_1 = 0;
    while ((x_1 < 5)) {
      let y_2 = 0;
      while ((y_2 < 5)) {
        console.log((x_1 * y_2));
        y_2 = (y_2 + 1);
        break;
      }
      x_1 = (x_1 + 1);
    }
  `,
},
{
  name: "functions",
  source: `
    let z = 0.5;
    function f(x: float, y: boolean) {
      print(sin(x) > π);
      return;
    }
```

```
    function g(): boolean {
      return false;
    }
    f(z, g());
  `,
  expected: dedent`
    let z_1 = 0.5;
    function f_2(x_3, y_4) {
      console.log((Math.sin(x_3) > Math.PI));
      return;
    }
    function g_5() {
      return false;
    }
    f_2(z_1, g_5());
  `,
},
{
  name: "arrays",
  source: `
    let a = [true, false, true];
    let b = [10, #a - 20, 30];
    const c = [[int]]();
    const d = random b;
    print(a[1] || (b[0] < 88 ? false : true));
  `,
  expected: dedent`
    let a_1 = [true,false,true];
    let b_2 = [10,(a_1.length - 20),30];
    let c_3 = [];
    let d_4 = ((a=>a[~~(Math.random()*a.length)])(b_2));
    console.log((a_1[1] || (((b_2[0] < 88)) ? (false) : (true))));
  `,
},
{
  name: "structs",
  source: `
    struct S { x: int }
    let x = S(3);
    print(x.x);
  `,
  expected: dedent`
    class S_1 {
    constructor(x_2) {
    this["x_2"] = x_2;
    }
    }
    let x_3 = new S_1(3);
```

```
    console.log((x_3["x_2"]));
  `,
},
{
  name: "optionals",
  source: `
    let x = no int;
    let y = x ?? 2;
    struct S {x: int}
    let z = some S(1);
    let w = z?.x;
  `,
  expected: dedent`
    let x_1 = undefined;
    let y_2 = (x_1 ?? 2);
    class S_3 {
    constructor(x_4) {
    this["x_4"] = x_4;
    }
    }
    let z_5 = new S_3(1);
    let w_6 = (z_5?.["x_4"]);
  `,
},
{
  name: "for loops",
  source: `
    for i in 1..<50 {
      print(i);
    }
    for j in [10, 20, 30] {
      print(j);
    }
    repeat 3 {
      // hello
    }
    for k in 1...10 {
    }
  `,
  expected: dedent`
    for (let i_1 = 1; i_1 < 50; i_1++) {
      console.log(i_1);
    }
    for (let j_2 of [10,20,30]) {
      console.log(j_2);
    }
    for (let i_3 = 0; i_3 < 3; i_3++) {
    }
```

```
      for (let k_4 = 1; k_4 <= 10; k_4++) {
      }
    `,
  },
  {
    name: "standard library",
    source: `
      let x = 0.5;
      print(sin(x) - cos(x) + exp(x) * ln(x) / hypot(2.3, x));
      print(bytes("∞§¶•"));
      print(codepoints("💪💪🖐👩🏾‍🎤"));
    `,
    expected: dedent`
      let x_1 = 0.5;
      console.log(((Math.sin(x_1) - Math.cos(x_1)) + ((Math.exp(x_1) * Math.log(x_1)) / Math
      console.log([...Buffer.from("∞§¶•", "utf8")]);
      console.log([...("💪💪🖐👩🏾‍🎤")].map(s=>s.codePointAt(0)));
    `,
  },
]

describe("The code generator", () => {
  for (const fixture of fixtures) {
    it(`produces expected js output for the ${fixture.name} program`, () => {
      const actual = generate(optimize(analyze(parse(fixture.source))))
      assert.deepEqual(actual, fixture.expected)
    })
  }
})
```

and the generator itself:

generator.js

```
// The code generator exports a single function, generate(program), which
// accepts a program representation and returns the JavaScript translation
// as a string.

import { voidType, standardLibrary } from "./core.js"

export default function generate(program) {
  // When generating code for statements, we'll accumulate the lines of
  // the target code here. When we finish generating, we'll join the lines
  // with newlines and return the result.
  const output = []

  const standardFunctions = new Map([
    [standardLibrary.print, x => `console.log(${x})`],
    [standardLibrary.sin, x => `Math.sin(${x})`],
```

```
  [standardLibrary.cos, x => `Math.cos(${x})`],
  [standardLibrary.exp, x => `Math.exp(${x})`],
  [standardLibrary.ln, x => `Math.log(${x})`],
  [standardLibrary.hypot, ([x, y]) => `Math.hypot(${x},${y})`],
  [standardLibrary.bytes, s => `[...Buffer.from(${s}, "utf8")]`],
  [standardLibrary.codepoints, s => `[...(${s})].map(s=>s.codePointAt(0))`],
])

// Variable and function names in JS will be suffixed with _1, _2, _3,
// etc. This is because "switch", for example, is a legal name in Carlos,
// but not in JS. So, the Carlos variable "switch" must become something
// like "switch_1". We handle this by mapping each name to its suffix.
const targetName = (mapping => {
  return entity => {
    if (!mapping.has(entity)) {
      mapping.set(entity, mapping.size + 1)
    }
    return `${entity.name}_${mapping.get(entity)}`
  }
})(new Map())

const gen = node => generators?.[node?.kind]?.(node) ?? node

const generators = {
  // Key idea: when generating an expression, just return the JS string; when
  // generating a statement, write lines of translated JS to the output array.
  Program(p) {
    p.statements.forEach(gen)
  },
  VariableDeclaration(d) {
    // We don't care about const vs. let in the generated code! The analyzer has
    // already checked that we never updated a const, so let is always fine.
    output.push(`let ${gen(d.variable)} = ${gen(d.initializer)};`)
  },
  TypeDeclaration(d) {
    // The only type declaration in Carlos is the struct! Becomes a JS class.
    output.push(`class ${gen(d.type)} {`)
    output.push(`constructor(${d.type.fields.map(gen).join(",")}) {`)
    for (let field of d.type.fields) {
      output.push(`this[${JSON.stringify(gen(field))}] = ${gen(field)};`)
    }
    output.push("}")
    output.push("}")
  },
  StructType(t) {
    return targetName(t)
  },
  Field(f) {
```

```
  return targetName(f)
},
FunctionDeclaration(d) {
  output.push(`function ${gen(d.fun)}(${d.params.map(gen).join(", ")}) {`)
  d.body.forEach(gen)
  output.push("}")
},
Variable(v) {
  // Standard library constants just get special treatment
  if (v === standardLibrary.π) return "Math.PI"
  return targetName(v)
},
Function(f) {
  return targetName(f)
},
Increment(s) {
  output.push(`${gen(s.variable)}++;`)
},
Decrement(s) {
  output.push(`${gen(s.variable)}--;`)
},
Assignment(s) {
  output.push(`${gen(s.target)} = ${gen(s.source)};`)
},
BreakStatement(s) {
  output.push("break;")
},
ReturnStatement(s) {
  output.push(`return ${gen(s.expression)};`)
},
ShortReturnStatement(s) {
  output.push("return;")
},
IfStatement(s) {
  output.push(`if (${gen(s.test)}) {`)
  s.consequent.forEach(gen)
  if (s.alternate?.kind?.endsWith?.("IfStatement")) {
    output.push("} else")
    gen(s.alternate)
  } else {
    output.push("} else {")
    s.alternate.forEach(gen)
    output.push("}")
  }
},
ShortIfStatement(s) {
  output.push(`if (${gen(s.test)}) {`)
  s.consequent.forEach(gen)
```

```
  output.push("}")
},
WhileStatement(s) {
  output.push(`while (${gen(s.test)}) {`)
  s.body.forEach(gen)
  output.push("}")
},
RepeatStatement(s) {
  // JS can only repeat n times if you give it a counter variable!
  const i = targetName({ name: "i" })
  output.push(`for (let ${i} = 0; ${i} < ${gen(s.count)}; ${i}++) {`)
  s.body.forEach(gen)
  output.push("}")
},
ForRangeStatement(s) {
  const i = targetName(s.iterator)
  const op = s.op === "..." ? "<=" : "<"
  output.push(`for (let ${i} = ${gen(s.low)}; ${i} ${op} ${gen(s.high)}; ${i}++) {`)
  s.body.forEach(gen)
  output.push("}")
},
ForStatement(s) {
  output.push(`for (let ${gen(s.iterator)} of ${gen(s.collection)}) {`)
  s.body.forEach(gen)
  output.push("}")
},
Conditional(e) {
  return `((${gen(e.test)}) ? (${gen(e.consequent)}) : (${gen(e.alternate)}))`
},
BinaryExpression(e) {
  const op = { "==": "===", "!=": "!==" }[e.op] ?? e.op
  return `(${gen(e.left)} ${op} ${gen(e.right)})`
},
UnaryExpression(e) {
  const operand = gen(e.operand)
  if (e.op === "some") {
    return operand
  } else if (e.op === "#") {
    return `${operand}.length`
  } else if (e.op === "random") {
    return `((a=>a[~~(Math.random()*a.length)])(${operand}))`
  }
  return `${e.op}(${operand})`
},
EmptyOptional(e) {
  return "undefined"
},
SubscriptExpression(e) {
```

```
      return `${gen(e.array)}[${gen(e.index)}]`
    },
    ArrayExpression(e) {
      return `[${e.elements.map(gen).join(",")}]`
    },
    EmptyArray(e) {
      return "[]"
    },
    MemberExpression(e) {
      const object = gen(e.object)
      const field = JSON.stringify(gen(e.field))
      const chain = e.op === "." ? "" : e.op
      return `(${object}${chain}[${field}])`
    },
    FunctionCall(c) {
      const targetCode = standardFunctions.has(c.callee)
        ? standardFunctions.get(c.callee)(c.args.map(gen))
        : `${gen(c.callee)}(${c.args.map(gen).join(", ")})`
      // Calls in expressions vs in statements are handled differently
      if (c.callee.type.returnType !== voidType) {
        return targetCode
      }
      output.push(`${targetCode};`)
    },
    ConstructorCall(c) {
      return `new ${gen(c.callee)}(${c.args.map(gen).join(", ")})`
    },
  }

  gen(program)
  return output.join("\n")
}
```

> ***Wait was that a real code generator?***
>
> *Perhaps you expected that we would first generate some kind of intermediate code, and then do code generation to a register-based machine like an x86 or MIPS, with all that flow analysis, static single assignment stuff, graph coloring for register allocation, and pipeline analysis.*
>
> *Well no, not today. When your compiler targets and existing language like JavaScript or C, you can simply rely on the fact that the authors of the JavaScript and C compiler did that work already. Alternatively, just output LLVM IR. It's more work, but there are advantages to doing so.*

At this point:

- ✅ Fill in *test/generator.test.js*
- ✅ Fill in *src/generator.js*.

- ✅ Run `npm test` and make sure we've satisfied the tests we wrote earlier. You should be at or near 100% test coverage.
- ✅ Run `node src/carlos.js examples/gcd.carlos js` and make sure the output looks good.

Now we can finish the compiler test

Remember the compiler test we started at the beginning of this module, in which we added a placeholder-test? It's time to fully test that `compile` function:

compiler.test.js

```
import { describe, it } from "node:test"
import assert from "node:assert/strict"
import compile from "../src/compiler.js"

const sampleProgram = "print(0);"

describe("The compiler", () => {
  it("throws when the output type is missing", () => {
    assert.throws(() => compile(sampleProgram), /Unknown output type/)
  })
  it("throws when the output type is unknown", () => {
    assert.throws(() => compile(sampleProgram, "no such type"), /Unknown output type/)
  })
  it("accepts the parsed option", () => {
    const compiled = compile(sampleProgram, "parsed")
    assert(compiled.startsWith("Syntax is ok"))
  })
  it("accepts the analyzed option", () => {
    const compiled = compile(sampleProgram, "analyzed")
    assert(compiled.kind === "Program")
  })
  it("accepts the optimized option", () => {
    const compiled = compile(sampleProgram, "optimized")
    assert(compiled.kind === "Program")
  })
  it("generates js code when given the js option", () => {
    const compiled = compile(sampleProgram, "js")
    assert(compiled.startsWith("console.log(0)"))
  })
})
```

At this point:

- ✅ Fill in *test/compiler.test.js*
- ✅ Run `npm test` and make sure we've satisfied the tests we wrote earlier. You should be at 100% coverage.

✅ git add, commit, and push.
✅ Celebrate!

Thanks!

Thanks for reading, and be sure to check out the entire project on GitHub.

You might feel like adding some new features to the Carlos language. Or enhancing the compiler. Or using it as a basis for a language *you* design and implement yourself.

Static Analysis

So, the parser says your program checks out according to the grammar. Doesn't mean it should be allowed to run, does it?

CONTENTS

Motivation • Examples of Contextual Constraints • Formal Specification of Context Rules • Implementing a Static Analyzer • Summary

Motivation

It's really hard to write a grammar that can determine all of the legality rules for a language. So in practice, we don't put too much effort into the grammar. We let the grammar take care only of **context-free** rules that can reject, for example:

```
clask C {int y/-&&^while var xyz += (((((((x;}
```

but leave rules like identifier uniqueness, type checking, parameter matching—and anything else that requires examining context—out of the grammar. So we say something like this:

```
class C {int y = x;}
```

is correct according to the grammar—some might even say it is syntactically correct. But the program still should not be allowed to run, as there is an error that can be detected by looking at the source code. Because the error is detectable before the program is executed, this is a **static** error, and finding these errors is part of the activity known as **static analysis**. Whether you call these kinds of errors "static semantic errors" or "context-sensitive syntax errors" is really up to you.

***Exercise**: Many language rules are not context free, and this can be proven. For example, you can indeed prove that enforcing the rule that the number of arguments in a call must match the number of parameters of the*

callee is not context-free. Prove this, or find, read, and understand an existing proof.

***Exercise**: Come up with some English sentences that are syntactically correct but semantically nonsense. Here are two to get you started: "Colorless green ideas sleep furiously" (Chomsky's famous example), and "One gaseous solid rocks smelling the color seventeen will think yesterday."*

In order to enforce the contextual constraints, it is necessary to **decorate** the parse tree or AST with contextual information.

> *Static analysis applies to some languages more than others. "Dynamic languages" do almost no analysis at compile time. Some languages are super static, permitting almost every check to be done before execution. These notes try to consider tons of possible semantic checks, whether or not most languages implement them.*

Examples of Contextual Constraints

The set of static constraints varies so much from language to language, so rather than focusing on a particular language, we'll just lay out a few that you might see somewhere:

- Identifiers have to be declared before they are used.
- Identifiers may not be used in a place where they may have not yet been initialized.
- Identifier declarations must be unique within a scope, unless a language provides multiple namespace buckets (e.g., functions vs non-functions), or it permits **overloading**.
- When assigning expression ***e*** to variable declared with type ***t*** (or passing argument ***e*** to a parameter ***v*** of type ***t***, or returning ***e*** from a function with return type ***t***), the type of ***e*** must be **compatible with** type ***t***. This may mean many things (that vary from language to language), but common rules are things like:
  - ***e***.type is the exact same type as ***t***, or
  - ***e***.type and ***t*** have the same name, or perhaps the same structure, or
  - ***e***.type is a **subtype** of ***t***, or
  - The programmer has set up an **implicit conversion**, or **coercion**, from ***e***.type to ***t***, or
  - ***e***.type can be **widened** to ***t***, e.g., byte→short→int→long→float→double... (though some languages notably disallow this!), or

    - e.type is a bottom type, or
    - t is a top type.

  Type checking also involves understanding overloading rules, the polymorphism mechanisms for the language, type inference rules, and how and when the language uses covariance, contravariance, invariance, and bivariance.

- Identifiers must be used according to (1) their **scope**, (2) their **access modifiers** (private, protected, package, public, whitelist, blacklist, etc.), and (3) any other meta-level attributes (enumerable, configurable, writable, deletable, callable, etc.). Sometimes scope rules are obvious. Sometimes both explicit import and explicit export is required.

- Arguments must match up with parameters in terms of number, order, name, mode, etc. Languages offer enormous flexibility here. Sometimes the number of arguments can be less or more than the number of parameters. Sometimes names are required for arguments and sometimes not.

- `break` and `continue` statements may only appear in a loop. `return` statements may only appear in a function. (It's *possible* to encode these restrictions in the grammar, but it would be ugly).

- All paths through a function must have a return.

- In an expression like `p.x`, p must have a dictionary type and the field x must be a field of the type of p. Or p is a module, package, or namespace, and x is an identifier marked as exportable from it.

- In an expression like `p[x]`, p must have an array type and x must have a type compatible with the index type of p's type.

- All possible matches in a pattern match expression, or a switch statement, must be covered.

- When inheriting from a base class with abstract methods, or implementing an interface, all abstract methods must be implemented, or the derived class must be declared abstract.

- All declared local variables must be subsequently read, and declared private functions must be called.

Formal Specification of Context Rules

It turns out that contextual constraints can be specified formally, just not in a context free grammar. Many techniques exist. Let's look at a couple.

Attribute Grammars

In an **attribute grammar**, we embed either **action routines** or **semantic functions** into a traditional (context-free) grammar. The idea is that each grammar variable can have attached to it any number of attributes (which can be whatever you like). When the grammar rules are being matched during a parse, the rules or actions fire, updating the attributes. Let's learn via example.

Example: Infix to Postfix Conversion

Here is an attribute grammar that produces a postfix string from an infix string, using **semantic functions**. We'll use this grammar for the source language (the infix):

```
E → E "+" T | E "-" T | T
T → T "*" F | T "/" F | F
F → "-" F | "(" E ")" | intlit | id
```

For the target language, we'll use " ~ " for unary negation in the postfix formulation in order to avoid parentheses. As long as all operators have a fixed arity, parentheses are not necessary.

Grammar Rules	Semantic Rules
E → E_1 "+" T	E.post := E_1.post • T.post • "+"
E → E_1 "−" T	E.post := E_1.post • T.post • "−"
E → T	E.post := T.post
T → T_1 "*" F	T.post := T_1.post • F.post • "*"
T → T_1 "/" F	T.post := T_1.post • F.post • "/"
T → F	T.post := F.post
F → "−" F_1	F.post := F_1.post • "~"
F → "(" E ")"	F.post := E.post
F → intlit	F.post := intlit.sourceString
F → id	F.post := id.sourceString

See how it works? Each symbol gets some properties (called attributes) as necessary, and we make rules that show how to assign attribute values. There's a lot of theory here that we won't cover, like whether

attributes are synthesized or inherited, but you should work on gaining a basic understanding of what attribute grammars look like.

CLASSWORK

Let's do some derivations.

Exercise: *Actually implement this thing in Ohm.*

Now here the specification using **action routines**. For these, it's best to change up the grammar just a bit.

```
E → T (addop T)*
T → F (mulop F)*
F → "-" F | "(" E ")" | intlit | id
```

Now our attribute grammar looks like this:

```
E     → T {E.post := T.post} (addop T {E.post •= T.post • addop.text})*
T     → F {T.post := F.post} (mulop F {T.post •= F.post • mulop.text})*
F     → - F1 {F.post := F1.post • "~"}
F     → "(" E {F.post := E.post} ")"
F     → id {F.post := image(id)}
F     → intlit {F.post := image(intlit)}
addop → "+" {addop.text := "+"} | "-" {addop.text := "-"}
mulop → "*" {mulop.text := "*"} | "/" {mulop.text := "/"}
```

Example: Polynomial Differentiation

An attribute grammar for finding the derivative of polynomials, using semantic functions:

Grammar Rules	Semantic Rules
P → P_1 + T	P.deriv := P_1.deriv • "+" • T.deriv
P → P_1 − T	P.deriv := P_1.deriv • "−" • T.deriv
P → T	P.deriv := T.deriv
P → − T	P.deriv := "−" • T.deriv
T → C	T.deriv := "0"
T → C x	T.deriv := C.val

Grammar Rules	Semantic Rules
T → C x ^ E	T.deriv := product(E.val,C.val) • "x^" • pred(E.val)
T → x	T.deriv := "1"
T → x ^ E	T.deriv := E.val • "x^" • pred(E.val)
C → const	C.val := valueof(const)
E → const	E.val := valueof(const)
E → - const	E.val := "-" • valueof(const)

With action routines:

```
P → {P.deriv := ""} ("-" {P.deriv := "-"})?
    T {P.deriv •= T.deriv}
    (
        ("+" {P.deriv •= "+"} |"-" {P.deriv •= "-"})
        T {P.deriv •= T.deriv}
    )*

T → {c := 1; e := 1} [N {c := N.value}]
    "x" ("^" ("-" {e := -1})? N {e *= -N.value})?
    {T.deriv := product(c,e) • "x^" • pred(e)}
  | N {T.deriv := "0"}
```

Note that Ohm feels a lot like writing attribute grammars with semantic functions. However Ohm allows arbitrary JavaScript code in its semantic functions, which is more flexible than just slapping attributes on to parse tree nodes.

The theory behind attribute grammars is pretty rich. In the compiler literature, much has been written about the order of attribute evaluation, and whether attributes bubble up the parse tree or can be passed down or sideways through the three. It's all fascinating stuff, and worthwhile when using certain compiler generator tools. But you can always just use Ohm and enforce contextual rules with code.

"Static Semantics"

Another approach is to just treat contextual rules as part of the **semantics** of a language, albeit not the same semantics that defines the runtime effects of a program. It's **static semantics**, and you can use the techniques of denotational or operational semantics to enforce the contextual rules, too.

For example, here's a way to define the contextual constraints of Astro. Here $\vdash p$ means program p is statically correct; $c \vdash e$ means expression e is correct in context c, and $c \vdash s \implies c'$ means that statement s is correct in context c and subsequent statements must be checked in context c'. In other words, statically analyzing a statement "updates" the context.

$$\frac{}{c \vdash [\![n]\!]} \qquad \frac{c(i) = (\mathsf{NUM}, _)}{c \vdash [\![i]\!]} \qquad \frac{c(i) = (\mathsf{FUN}, n) \quad (c \vdash e_i)_{i=1}^{n}}{c \vdash [\![i\ e_1 \ldots e_n]\!]}$$

$$\frac{c \vdash e}{c \vdash [\![-e]\!]} \qquad \frac{op \in \{+, -, *, /, \%, **\} \quad c \vdash e_1 \quad c \vdash e_2}{c \vdash [\![e_1\ op\ e_2]\!]} \qquad \frac{c \vdash e}{c \vdash [\![\mathtt{print}\ e]\!] \implies c}$$

$$\frac{c \vdash e \quad c(i) = \mathsf{UNDEF} \vee c(i) = (\mathsf{NUM}, \mathsf{RW})}{c \vdash [\![i = e]\!] \implies c[(\mathsf{NUM}, \mathsf{RW}) \,/\, i]} \qquad \frac{(c_{i-1} \vdash s_i \implies c_i)_{i=1}^{n}}{\vdash [\![\mathtt{program}\ s_1 \ldots s_n]\!]}$$

$$\begin{aligned} c_0 = \lambda i.\, &\mathsf{UNDEF}\ [\\ &(\mathsf{NUM},\ \mathsf{RO}) \,/\, \pi]\ [\\ &(\mathsf{FUN}, 1) \,/\, \mathtt{sqrt}]\ [\\ &(\mathsf{FUN}, 1) \,/\, \mathtt{sin}]\ [\\ &(\mathsf{FUN}, 1) \,/\, \mathtt{cos}]\ [\\ &(\mathsf{FUN}, 2) \,/\, \mathtt{hypot}] \end{aligned}$$

Previously, we gave formal definitions of Astro and Bella in which static and dynamic semantics were defined together. If we do decide to make a static semantics on its own, then the dynamic semantics can become simpler, since we can assume all the static checks have already been done. This is, in fact, how real compilers and interpreters work.

***Exercise**: Define a new dynamic-only operational semantics of Astro, assuming the static semantics above.*

***Exercise**: Split the semantics of Bella into static and dynamic parts.*

Implementing a Static Analyzer

Logically speaking we do static analysis by traversing the CST or AST, decorating it, and checking things. We do quite a few tasks here, such as name and type resolution, control flow analysis, and data flow analysis. These tasks are often interleaved.

Name and Type Resolution

First we figure out which names refer to which (declared) entities, and what the types are for each expression. The first part uses is sometimes called scope analysis and involves **symbol tables** and the second does (some degree of) **type inference**.

Symbol Tables

A **symbol table** is a collection of mappings from names (identifiers) to entities. That is all.

There is a design thing to be aware of, though. Should you (1) create a new symbol table for every scope, or (2) manage just one symbol table? What are the tradeoffs?

	Table per Scope	**Single Table**
Basic Idea	There is a symbol table for every scope. Maps identifier to (usually a) single entity.	There is only one table. Maps identifier to *list of entities.*
On scope entry	Create a new, empty symbol table. Set this table's parent to the current table.	Nothing really. Or you could increment a counter so you always have a nesting level associated with each entity.
On a declaration	Add identifier/entity pair to table.	*Push* entity onto list for the identifier.
On a use	Search current table; if not found, recursively search the parent table (and so on).	Search the list.
On scope exit	Nothing, really.	*Pop* all entities added in this scope.
Advantages	Easy to implement. Tables can be immutable.	Lookup is efficient.
Disadvantages	Search can be slow, especially for globals!	Mutable. Scope exit expensive.

Type inference and type checking

So how do we determine (infer) the type of every expression? This can get interesting. It depends on the language design. Statically typed languages generally fall into a spectrum of how much type inference they allow:

How much?	Characteristics	Exemplars
Low	Very verbose. Pretty much every declaration (of every variable, every field, every parameter, every function) mentions its type.	Java, prior to version 10
Medium	Generally variable declarations don't need types (you can say `var x = 100` or `var message = "Hello"`), but parameter types and function return types must be explicit.	Go, Swift, Rust, C#
High	Virtually every identifier declaration can be given without a type. The inference engine has to do a ton of work. But the code looks so pretty and light, and we still get type safety guarantees before execution! Often uses a Hindley-Milner type system (with type variables) and perhaps, type classes.	Standard ML, OCaml, F#, Haskell, Elm

***Exercise**: Read about the Hindley-Milner type system.*

So how to do type checking? You follow the type-equivalence and type-compatibility rules. These vary from language to language and are generally very straightforward, until you get to parameterized types, or generics. Why are these not straightforward? Well suppose `Dog` is a subclass of `Animal`:

```
Dog <: Animal
```

Now what is the relationship between `List<Dog>` and `List<Animal>`? There are four cases:

- `List<Dog> <: List<Animal>` — covariance
- `List<Animal> <: List<Dog>` — contravariance
- Both are subclasses of the other — bivariance
- Neither are subclasses of the other — invariance

Then it gets more complicated. Do you remember, offhand, the relationships between `List<Dog>`, `List<Animal>`, and:

```
List<? extends Dog>
List<? extends Animal>
List<? super Dog>
List<? super Animal>
List<?>
List<Object>
```

***Exercise**: Draw the class inheritance graph for these classes.*

Control Flow Analysis

Control Flow Analysis (CFA) is what we do when we build and query the control flow graph (CFG). This can help us find functions that are never called, code that is unreachable, some infinite loops, paths without return statements, etc.

Read about CFA and CFGs at Wikipedia.

Data Flow Analysis

In DFA, we determine where identifiers are declared, when they are initialized, when they are updated, and who reads (refers to) them. This tells us when identifiers are used but not declared, used but not initialized, declared but never used, etc. Also we can note for each identifier at each point in the program, which other entities *could* refer to them.

Read about DFA at Wikipedia.

Languages like Rust also do ownership and borrowing computations as part of data flow analysis. We *might* cover this in class. If we don't read these three sections from *The Rust Programming Language Book*: Ownership, References and Borrowing, and Lifetimes.

Linting

A complier's static analyzer only *needs* to check whether programs violate language rules. But there are also many such statically "correct" programs that are written weirdly, extremely error prone, under-performant, resource-leaking, subject to race conditions, or produce completely unexpected (in other words, wrong) results when run. These problems can be captured by a **linter**.

Look at these cool lists:

- List of (potential) Java bugs detectable by FindBugs™.
- List of (potential) Java bugs detectable by PMD.
- List of (potential) JavaScript bugs detectable by PMD.
- The rules of ESLint.

You should integrate a linter into your text editor or IDE. Seeing both language errors (from the compiler) and linter errors while you write your program is a Good Thing.

> ***Exercise**: Skim these lists. Which rules are your favorites? Which surprised you?*

***Exercise**: Find some linters for Python, Ruby, PHP, and C#.*

Summary

We've covered:

- ✅ Examples of Contextual Constraints
- ✅ Formal Specification of Context Rules
- ✅ Attribute Grammars
- ✅ Static semantic rules
- ✅ Implementing a Static Analyzer
- ✅ Linting

Code Generation

What about the back half of the compiler?

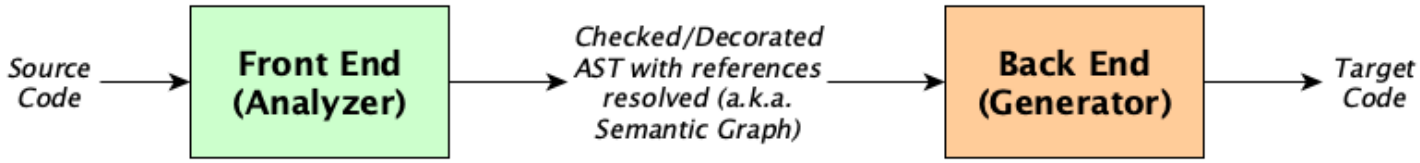
CONTENTS

What is Code Generation? • Goals of Code Generation • What Kind of Code Can We Generate? • Two More Questions • Transpiling • Virtual Machines and Backend Language Libraries • Targeting a Typical Processor • Code Generator Generators • Summary

What is Code Generation?

The first part of a compiler **analyzes** the source code into a structure that carries the meaning of the program; this structure is generally the abstract syntax tree that's been checked and decorated. (Remember decorated means all identifier references have been resolved.)

From this structure we can **generate** the corresponding code in some other language, the target language. This is what a code generator does.



Some compilers generate twice: they first generate code in some "intermediate language" like SIL, LLVM IR, HIR, MIR, CIL, etc. *Then* they do the "real" code generation into a target language that is directly runnable (or really close to it), like virtual machine code, assembly language, or machine language.



Goals of Code Generation

Here are the three goals of any good code generator:

Correctness

It should produce correct code

Efficiency

It should produce efficient code

Self-Efficiency

It should itself be efficient

***Exercise**: The last two goals are seemingly in conflict. Discuss.*

What Kind of Code Can We Generate?

There is no one way to do code generation. There is no one target of a code generator. A code generator can produce:

- **Code in another high-level language**. Don't knock it. If the target high-level language has a good compiler already, this is a great strategy. Stand on the shoulders of giants right?
- **C**. Oh wait, C is a high-level language. Sort of. Wait no. No lambdas. But then you can let the C compiler take over to do the rest.
- **An intermediate representation** like LLVM, that has backends for dozens of architectures.
- **Executable virtual machine code**, such as for the JVM or BEAM. Targeting a virtual machine is a ton of fun. Learning about a VM and its instruction set is a great experience. You'll get a great feeling of accomplishment. And you can run this code, too.
- **Assembly language**: Because it's so fulfilling to learn about machine architecture, especially pipelines, caches, and of course, registers! You need to know your machine. Is it a RISC or CISC processor or some kind of novel architecture like a dataflow processor or somthing with hundreds of cores? What are the

relative instruction speeds? How are its addressing modes? Does it have out-of-order instruction? Any cool idioms?

- **Relocatable machine language**: you've done a lot of assembly that the assembler would have done for you (resolving symbols), but at least you will let the linker do the rest.
- **Absolute machine language**: you've resolved all the memory locations; in other words you spent a lot of time doing what assemblers and linkers would do for you already.

Are there other kinds of code?

Two More Questions

Also consider:

- Should we generate awesome code on the fly? Or let a separate optimizing pass clean up after our generator? Or not optimize at all? For example, naive code generation can lead to not so great code:

```
x = x + 1;        mov rax, x_0     ┐
                  add rax, 1       ├     just need inc x_0
                  mov x_0, rax     ┘

x = y + c;        mov rax, y_5
z = x + 4;        add rax, c_8
                  mov x_2, rax     ; Redundant if x not used again
                  mov rax, x_2     ; Redundant
                  add rax, 4
                  mov z_7, rax
```

- Should we stick in line number information so a debugger can take over if our program is stopped intentionally (or crashes)?

Transpiling

JavaScript makes a great target language? Why? It's got first-class functions and async support, and that Node.js thing runs on that awesome V8. And the ecosystem! How can we target JS?

Start by browsing an existing example. This one, for the language Carlos, translates the decorated AST (Semantic Graph) directly into JavaScript (there's no need to use an IR if you are targeting a high-level language):

generator.js

```
// The code generator exports a single function, generate(program), which
// accepts a program representation and returns the JavaScript translation
// as a string.

import { voidType, standardLibrary } from "./core.js"

export default function generate(program) {
  // When generating code for statements, we'll accumulate the lines of
  // the target code here. When we finish generating, we'll join the lines
  // with newlines and return the result.
  const output = []

  const standardFunctions = new Map([
    [standardLibrary.print, x => `console.log(${x})`],
    [standardLibrary.sin, x => `Math.sin(${x})`],
    [standardLibrary.cos, x => `Math.cos(${x})`],
    [standardLibrary.exp, x => `Math.exp(${x})`],
    [standardLibrary.ln, x => `Math.log(${x})`],
    [standardLibrary.hypot, ([x, y]) => `Math.hypot(${x},${y})`],
    [standardLibrary.bytes, s => `[...Buffer.from(${s}, "utf8")]`],
    [standardLibrary.codepoints, s => `[...(${s})].map(s=>s.codePointAt(0))`],
  ])

  // Variable and function names in JS will be suffixed with _1, _2, _3,
  // etc. This is because "switch", for example, is a legal name in Carlos,
  // but not in JS. So, the Carlos variable "switch" must become something
  // like "switch_1". We handle this by mapping each name to its suffix.
  const targetName = (mapping => {
    return entity => {
      if (!mapping.has(entity)) {
        mapping.set(entity, mapping.size + 1)
      }
      return `${entity.name}_${mapping.get(entity)}`
    }
  })(new Map())

  const gen = node => generators?.[node?.kind]?.(node) ?? node

  const generators = {
    // Key idea: when generating an expression, just return the JS string; when
    // generating a statement, write lines of translated JS to the output array.
    Program(p) {
```

```
  p.statements.forEach(gen)
},
VariableDeclaration(d) {
  // We don't care about const vs. let in the generated code! The analyzer has
  // already checked that we never updated a const, so let is always fine.
  output.push(`let ${gen(d.variable)} = ${gen(d.initializer)};`)
},
TypeDeclaration(d) {
  // The only type declaration in Carlos is the struct! Becomes a JS class.
  output.push(`class ${gen(d.type)} {`)
  output.push(`constructor(${d.type.fields.map(gen).join(",")}) {`)
  for (let field of d.type.fields) {
    output.push(`this[${JSON.stringify(gen(field))}] = ${gen(field)};`)
  }
  output.push("}")
  output.push("}")
},
StructType(t) {
  return targetName(t)
},
Field(f) {
  return targetName(f)
},
FunctionDeclaration(d) {
  output.push(`function ${gen(d.fun)}(${d.params.map(gen).join(", ")}) {`)
  d.body.forEach(gen)
  output.push("}")
},
Variable(v) {
  // Standard library constants just get special treatment
  if (v === standardLibrary.π) return "Math.PI"
  return targetName(v)
},
Function(f) {
  return targetName(f)
},
Increment(s) {
  output.push(`${gen(s.variable)}++;`)
},
Decrement(s) {
  output.push(`${gen(s.variable)}--;`)
},
Assignment(s) {
```

```
  output.push(`${gen(s.target)} = ${gen(s.source)};`)
},
BreakStatement(s) {
  output.push("break;")
},
ReturnStatement(s) {
  output.push(`return ${gen(s.expression)};`)
},
ShortReturnStatement(s) {
  output.push("return;")
},
IfStatement(s) {
  output.push(`if (${gen(s.test)}) {`)
  s.consequent.forEach(gen)
  if (s.alternate?.kind?.endsWith?.("IfStatement")) {
    output.push("} else")
    gen(s.alternate)
  } else {
    output.push("} else {")
    s.alternate.forEach(gen)
    output.push("}")
  }
},
ShortIfStatement(s) {
  output.push(`if (${gen(s.test)}) {`)
  s.consequent.forEach(gen)
  output.push("}")
},
WhileStatement(s) {
  output.push(`while (${gen(s.test)}) {`)
  s.body.forEach(gen)
  output.push("}")
},
RepeatStatement(s) {
  // JS can only repeat n times if you give it a counter variable!
  const i = targetName({ name: "i" })
  output.push(`for (let ${i} = 0; ${i} < ${gen(s.count)}; ${i}++) {`)
  s.body.forEach(gen)
  output.push("}")
},
ForRangeStatement(s) {
  const i = targetName(s.iterator)
  const op = s.op === "..." ? "<=" : "<"
```

```
    output.push(`for (let ${i} = ${gen(s.low)}; ${i} ${op} ${gen(s.high)}; ${i}++) {`)
    s.body.forEach(gen)
    output.push("}")
  },
  ForStatement(s) {
    output.push(`for (let ${gen(s.iterator)} of ${gen(s.collection)}) {`)
    s.body.forEach(gen)
    output.push("}")
  },
  Conditional(e) {
    return `((${gen(e.test)}) ? (${gen(e.consequent)}) : (${gen(e.alternate)}))`
  },
  BinaryExpression(e) {
    const op = { "==": "===", "!=": "!==" }[e.op] ?? e.op
    return `(${gen(e.left)} ${op} ${gen(e.right)})`
  },
  UnaryExpression(e) {
    const operand = gen(e.operand)
    if (e.op === "some") {
      return operand
    } else if (e.op === "#") {
      return `${operand}.length`
    } else if (e.op === "random") {
      return `((a=>a[~~(Math.random()*a.length)])(${operand}))`
    }
    return `${e.op}(${operand})`
  },
  EmptyOptional(e) {
    return "undefined"
  },
  SubscriptExpression(e) {
    return `${gen(e.array)}[${gen(e.index)}]`
  },
  ArrayExpression(e) {
    return `[${e.elements.map(gen).join(",")}]`
  },
  EmptyArray(e) {
    return "[]"
  },
  MemberExpression(e) {
    const object = gen(e.object)
    const field = JSON.stringify(gen(e.field))
    const chain = e.op === "." ? "" : e.op
```

```
      return `(${object}${chain}[${field}])`
    },
    FunctionCall(c) {
      const targetCode = standardFunctions.has(c.callee)
        ? standardFunctions.get(c.callee)(c.args.map(gen))
        : `${gen(c.callee)}(${c.args.map(gen).join(", ")})`
      // Calls in expressions vs in statements are handled differently
      if (c.callee.type.returnType !== voidType) {
        return targetCode
      }
      output.push(`${targetCode};`)
    },
    ConstructorCall(c) {
      return `new ${gen(c.callee)}(${c.args.map(gen).join(", ")})`
    },
  }

  gen(program)
  return output.join("\n")
}
```

What did you notice from browsing these examples? A few tips and tricks, right? Things like:

- You write a `generate` method for each AST node, and each node's `generate` method invokes the generate method of its sub nodes. The top-level function `gen` **dispatches** to the right generate method
- When writing out high-level code, it is good to over-parenthesize: you should probably not assume your associativities and precedences and arities and fixities are the same as those of your target language.
- We should decorate the ids in the target language with uniquely identifying suffixes, because (1) the set of reserved words may differ between source and target languages, and (2) you never know whether scope rules are the same!
- While not shown in the examples above, character sets may very well be different between the two languages; keep that in mind.
- Inline some standard functions here. (But where standard functions cannot be inlined, your code generator can always write out the function implementation in the target language.)
- If your source language has static type checking, static access checks, static matching checks, static read-only checks, safe optionals, etc., these have already been done in the analyzer, and do not need to be handled in the generator! The Carlos generator above is an example of this.
- However, if your source language is dynamic, then you must generate code to do the type, access, read-only, null, and matching checks!

- Even for very static languages, some checks will still have to be done in generated code, like array length checks (when the array is a variable whose size is unknown until run-time)
- There may be a ton of ad-hoc things that pop up, but these are fun challenges.

Here is a view of some of the Carlos functions in table form:

Graph Fragment	JavaScript
program / \ s_1 .. s_n	s_1'; . . s_n';
vardecl / | \ v T E	let v' = E'

structtypedecl
/ \
t structtype
/ \
field .. field
/ \ / \
x_1 T_1 x_n T_n
	class t' {
	constructor (x_1', .., x_n') {
	this.x_1' = x_1';
	...
	this.x_n' = x_n';
	}
	}

functiondecl
/ | \
function T B
/ | \
f param .. param
/ \ / \
x_1 T_1 x_n T_n
	function f(x_1', .., x_n') {
	B
	}

inc | V	V'++
dec | V	V'--
assign / \ V E	V' = E'
break	break
return	return

return | E	return E'
if / \ E B	if (E') { B' }
if / | \ E B_1 B_2	if (E') { B_1' } else { B_2'} // But no braces around B_2 if it's an if-statement
while / \ E B	while (E') { B' }
repeat / \ E B	for (let i = 0; i < E'; i++) { B' } // (note: i is a new variable)
for-upto / | | \ x E_1 E_2 B	for (let x' = E_1'; x' < E_2'; x'++) { B' }
for-through / | | \ x E_1 E_2 B	for (let x' = E_1'; x' <= E_2'; x'++) { B' }
for-collection / | \ x A B	for (let x' of A') { B' }
conditional / | \ E E_1 E_2	((E') ? (E_1') : (E_2'))

+
/ \
E_1 E_2
	(E_1' + E_2)
	// same for all other binary ops
	// translated where necessary (e.g. == → ===)

-
|
E
	(- E')
	// same for all other unary ops
	// except "some", which is a no-op in JS

emptyoptional	undefined
subscript / \ A E	A'[E']
arrayexpression / \ E_1 ..... E_n	[E_1', ..., E_n']

member / \ O F	O'["F'"] // quotes are important
fn-call / | \ F E_1 .. E_n	F'(E_1', ..., E_n') // Standard Library functions handled inline
ctor-call / | \ C E_1 .. E_n	new C'(E_1', ..., E_n')

Virtual Machines and Backend Language Libraries

LLVM

LLVM is, well, it's huge, it's impressive, and lots of compilers for modern languages are using it. It is a library with a ton of stuff, including an intermediate language and backends. It's official name is **The LLVM Compiler Infrastructure Project**.

A take on modern compiler construction from Andi McClure:

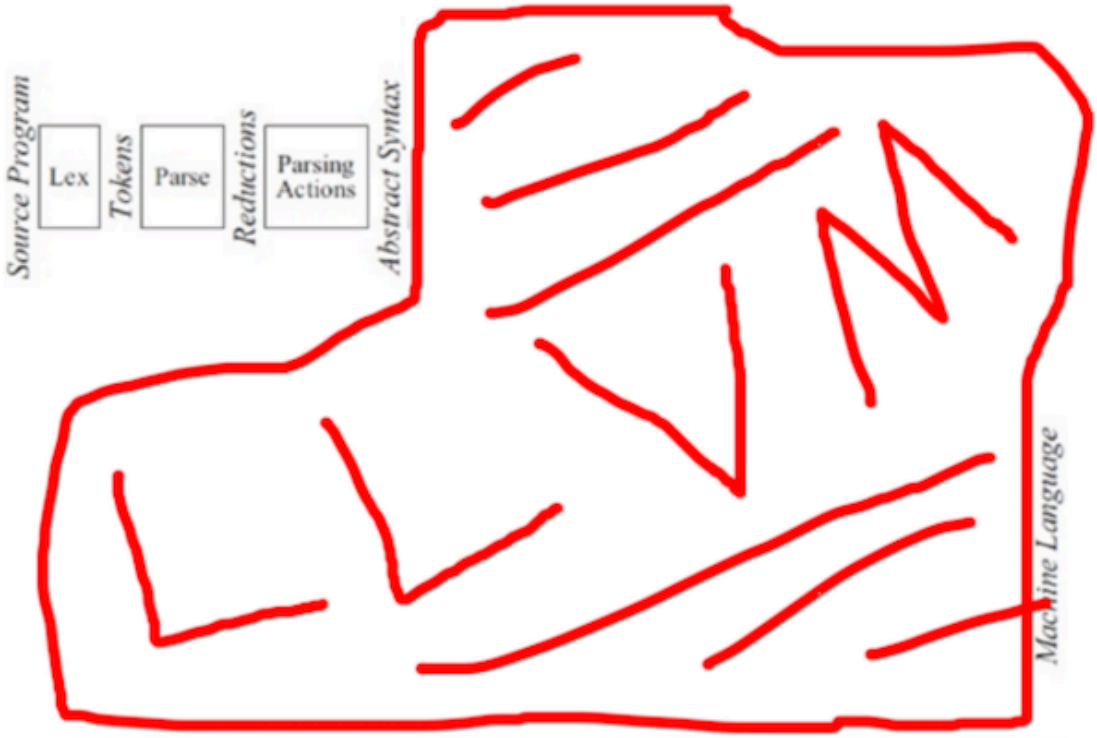


FIGURE 1.1. Phases of a compiler, and interfaces between them

***Exercise**: Read the whole blog post from whence came that awesome picture.*

JVM

Read about the JVM at Wikipedia and about its formal specification.

Web Assembly

Read about Web Assembly

***Exercise**: Explain WASM, briefly, in your own words.*

Write your own VM!

SERIOUSLY. DO IT.

Targeting a Typical Processor

Okay let's say you're going to bypass LLVM or anything else and you are going straight to assembly. Or, let's say you are joining the LLVM team and want to enhance the project. Or you want to write a new backend for an intermediate representation. Lots of good stuff to know. Here are some notes that just scratch the surface of generating assembly language.

Address Assignment

TODO - Fortran no recursion local vars, even return val on stack

TODO - C, show assembly for recursive function

TODO - C, show space for local vars

TODO - motivate stack frames

TODO - motivate dynamic links

TODO - Three approaches to static scope

TODO - Static link

TODO - Single display

TODO - Display in every frame

TODO - No need for display or static link in leaf subroutine

TODO - Who saves the registers

TODO - Inlining?

TODO - Summary of questions and concerns

Instruction Selection

TODO

Register Allocation

TODO

Code Generator Generators

TODO

Summary

We've covered:

- ✅ -
- ✅ -